CITYPULSE

AI-Powered Urban Intelligence & Blockchain Analytics

1.0 - technical implementation report

Engineering project; synthetic demonstration; no production certification

0.1.0 / API 0.1.0-SNAPSHOT

manikantbindass (verified GitHub repository owner)

https://github.com/manikantbindass/CityPulse

Source snapshot: 2385ea7a3c6f

Evidence date: 2026-10-29

Contents

01 Executive summary 4

02 Abstract and introduction 5

03 Problem statement, motivation and objectives 5

04 Existing approaches and gap analysis 6

05 Scope, users and requirements 6

06 Technology choices and tradeoffs 7

07 Complete system architecture 8

08 Backend components and request lifecycle 10

09 Frontend architecture and accessible product behavior 12

10 Backend contracts, errors and implemented behavior 13

11 Database, PostGIS and entity relationships 16

12 Data sources, licensing and retention semantics 17

13 Ingestion, quality and idempotent publication 18

14 Supply-demand analytical methodology 21

15 AI forecasting methods and feature engineering 22

16 Training protocol and actual evaluation results 24

17 Blockchain design and proof verification 27

18 PulseAI analyst, retrieval and safety controls 30

19 Opportunity discovery and resource workspace 35

20 Optimization algorithms and measured cases 37

21 Authentication, authorization and security architecture 38

22 STRIDE threat modeling 42

23 Illustrative control mapping, not certification 44

24 UI/UX and real application screenshots 44

25 Infrastructure, cloud and progressive delivery 50

26 Observability, monitoring and proposed SLOs 52

27 Performance, scaling and cost assumptions 55

28 Reliability and disaster recovery 57

29 Account/key operations and incident response 58

30 Testing strategy and real results 60

31 Demonstration scenarios and business applications 61

32 Current limitations, future enhancements and conclusion 62

33 Appendix A - API inventory 62

34 Appendix B - physical data dictionary 65

35 Appendix C - architecture decision records 73

36 Appendix D - commands, configuration and glossary 75

37 References, attributions and reproducibility 77

How to read this evidence report

IMPLEMENTED means code exists at the source snapshot. DEMONSTRATED means a cited local/CI experiment ran. PROPOSED means a future target or operational design, not achieved behavior. Synthetic measurements remain labeled throughout.

The source hierarchy is code/migrations, measured JSON and revision-linked workflow evidence, then current operational documentation. Earlier project prose is historical where it contradicts inspected code. A source-hash manifest and editable Markdown/JSON support regeneration.

No legal author name, institutional affiliation, student identity, production deployment, live feed, public explorer, guaranteed investment outcome, external audit or certification is inferred. Repository ownership is the verified GitHub account. No project license has been selected.

Navigation: the table of contents and PDF bookmarks link to numbered chapters. Tables repeat headers; charts state units, scope and limitations. The appendices inventory actual route declarations and schema text rather than guessed endpoints.

01 Executive summary

CityPulse integrates urban supply-demand investigation with explicit geography, provenance, forecasting, constrained optimization and selected blockchain attestations. Its original concept is to understand city supply and demand through AI and blockchain. The implemented system combines a Next.js workspace, modular Java/Spring API, isolated Python service, PostgreSQL/PostGIS, Redis, an ingestion CLI and locally tested Solidity registries.

The contribution is an inspectable engineering platform rather than a new forecasting algorithm, an operational municipal deployment or a measured urban intervention. Sample dashboards and reproducible experiments are synthetic. Real HTTP integration and persistence are exercised in Linux CI, while the local browser can run an explicitly labeled unsaved SciPy scenario. Source licenses, real provider completeness and operational acceptance remain separate prerequisites.

The inspected baseline includes 29 Java tests, 48 AI tests, 13 web unit tests, 18 desktop/mobile browser tests, six ingestion tests and four Solidity tests with supporting CI evidence at the recorded revisions. Two mocked Terraform plans check infrastructure safety properties. Compose verifies real service requests, persistence, idempotent publication, runtime DB grants, metric scrapes and actual trace ingestion. These are bounded engineering checks, not penetration testing, an availability history or compliance certification.

The machine-learning evaluation covers eight synthetic city/district/category segments. Chronological validation selects among seasonal-naive, moving average, Ridge and XGBoost; untouched test results are retained per candidate. Residual-based intervals and recursive widening are explicitly diagnostic. The selected candidate is not necessarily best on the held-out test period. Synthetic error does not establish real-city accuracy.

A k6 arrival workload sent 21 real synthetic decision requests over 20 seconds directly to the updated FastAPI service. HTTP failures were zero and p95 request duration was 74.68 ms. The measurement excludes browser, Java, database and external-network latency; it is not a production throughput claim. The solver reports infeasible, unsupported and insufficient-data states instead of manufacturing allocations.

Security uses configured OIDC/PKCE, encrypted server sessions, active membership narrowing, tenant/owner predicates, strict request limits, minimized logs and restricted runtime database privileges. Actual external-provider enrollment, operational MFA, independent security review and restore drills remain unverified. A matching chain digest demonstrates byte integrity, not source truth. AWS infrastructure is validated code, not a deployed environment; AI persistence remains single-writer.

Implementation sources: README.md; docs/PRD.md

02 Abstract and introduction

Urban resource analysis requires reconciling quantities across geography, time, category and source coverage. CityPulse explores an architecture that keeps these dimensions explicit while connecting analytical, forecasting, optimization and verification workflows. The project evaluates software behavior and reproducible computational experiments, rather than asserting policy effectiveness or commercial success.

The methodological source hierarchy is deliberate: executable code and migrations govern implementation claims; structured evaluation files govern measured results; workflow logs establish which checks actually ran; later operational documents describe unverified deployment gates. Earlier PRD, frontend and architecture drafts contain obsolete planned/implemented statements and cannot override newer code. This report curates those contradictions instead of reproducing them as current facts.

The engineering questions are whether an authorized analyst can reconstruct a scenario from canonical inputs and fingerprints; whether missing observations remain explicit; whether a solver preserves hard policy constraints when infeasible; whether retrieved instructions can introduce unsupported model claims; and whether another party can compare exported bytes with a recorded digest. Tests support bounded answers, while field evaluation and operational assurance remain future work.

Keywords: urban analytics; geospatial provenance; chronological evaluation; tenant isolation; constrained allocation; evidence-grounded assistant; content attestation; reproducible deployment.

Implementation sources: README.md; docs/PRD.md

03 Problem statement, motivation and objectives

A district can show demand while another holds inventory, but combining those numbers is meaningful only if units, commodity, interval and coverage agree. Business-location density is not service capacity; inventory is not a flow; an absent supply report is not confirmed zero supply. A forecast can conceal rather than solve this measurement problem if its uncertainty and training provenance disappear in a dashboard.

CityPulse makes these distinctions part of data and workflow. Observations separate event time from ingestion time, retain source/mode/schema metadata and use bounded measurement units. Geographic containment and tenant-safe references constrain storage. Server-derived filters and ownership rules determine retrievable evidence. Human reviewers can inspect assumptions before accepting a recommendation; no optimization result dispatches resources.

Objectives are to preserve traceable observations, compare compatible evidence, evaluate forecasting against transparent baselines, formulate constrained resource scenarios, explain uncertainty and support independent integrity checks. Supporting objectives include finite request/compute budgets, explicit failure states, accessible interaction and a progressive infrastructure path without automatic cloud expenditure.

The practical motivation is reconstructability: an analyst should be able to identify the source records, model version, business assumptions and constraints behind a result. This is a software design objective. The project has not measured reductions in municipal cost, improved access, business profit or real-world forecasting error.

Implementation sources: README.md; docs/PRD.md

04 Existing approaches and gap analysis

Spreadsheets, GIS applications, forecasting notebooks, optimization scripts and audit ledgers are valid tools within their domains. Their integration can lose semantics: a notebook forecast may be copied without its training period, a dashboard may combine unequal units, and an exported recommendation may omit the policy assumptions that produced it. This is a qualitative engineering gap analysis, not a benchmark or competitive-product feature survey.

CityPulse addresses these transition points through typed requests, provenance fields, canonical snapshots, versioned computation and protected retrieval. Its assistant returns only verified statement selections and citations constructed by the API. A digest receipt is separated from an independently confirmed chain receipt. The user can inspect underlying assumptions rather than infer certainty from visual polish.

The proposed solution is intentionally modular. The browser explores, the API authorizes and persists, the numerical service names models and constraints, and the blockchain records selected attestations. Introducing Kafka, Kubernetes or additional model families without measured need would not itself resolve missing data, licensing or organizational ownership. Those remain pilot acceptance requirements.

Implementation sources: README.md; docs/PRD.md

05 Scope, users and requirements

The demonstrated scope is a local/CI platform with one shared web app, a modular API, isolated Python computation, an operator ingestion CLI and local EVM tests. Admin and reporting features are existing UI/API boundaries; reserved directories do not imply separately deployed services. Billing, self-service tenant signup, autonomous dispatch, licensed live city partnerships and a public mainnet service are not demonstrated.

Actor	Use case	Authorization boundary
Citizen	Inspect permitted registry context	Registry scope; no implicit access to private analytics
Analyst	Compare records, forecasts and scenarios	Active membership, scopes and owned decision objects

Actor	Use case	Authorization boundary
Planner	Review budget, capacity and fairness scenarios	Human-reviewed recommendations, no dispatch
Business owner	Investigate tenant-authorized business data	General business rows are tenant-scoped, not individually owner-scoped
Administrator	Provision memberships and inspect audit	Server policy; cannot remove final active administrator
Data provider	Publish licensed source observations	Strict validation, provenance and idempotent writes

Functional acceptance means observable behavior: invalid geometry/units are rejected, protected lists are bounded, retried writes preserve identity, models disclose versions/limits, missing evidence is not invented, scenarios can be saved/exported/reproduced and foreign owned objects do not disclose content. Later chapters map these requirements to code, routes, demonstrations and tests.

Nonfunctional requirements include visible keyboard focus, responsive layouts, reduced motion, bounded parser/body/solver inputs, private identity boundaries, minimized logs and reproducible builds. Operations proposes 99.5% eligible-request availability, sub-500 ms p95 registry reads at validated load and sub-two-second small computation latency. Earlier PRD targets differ; all remain proposed until measured under a defined workload and acceptance policy.

Implementation sources: README.md; docs/PRD.md

06 Technology choices and tradeoffs

Choice	Rationale	Material tradeoff
Next.js / React / TypeScript	Routing, typed interaction and server adapters	Dynamic nonce CSP changes rendering/caching assumptions
Spring Boot 4 / Java 21	Validation, authorization, transactions and migration discipline	Temporary Jackson 2 bridge is deprecated and needs later migration
FastAPI / SciPy / scikit-learn	Numerical isolation, validated requests and inspectable artifacts	SQLite/files constrain distributed execution
PostgreSQL / PostGIS	Transactional spatial and temporal integrity	Projection, containment and coverage need explicit semantics
Redis	Shared sessions, short cache TTL and request admission	Failure must not bypass security checks
Solidity / OpenZeppelin	Role-controlled digest attestations	Gas, keys, finality and immutable-data limitations
Compose then ECS	Reproducible integration before managed deployment	Mock plans do not verify real AWS permissions or service health

The architecture favors modular simplicity over premature service fragmentation. Java owns canonical business operations; Python isolates numerical dependencies; the web layer exposes

interaction and protects adapters. ADRs record domain boundaries, geographic integrity, measured model selection and delayed event infrastructure. ADR 009 adds the managed-cloud path and explicitly retains a single AI writer.

No broker is deployed merely for an event-driven label. Optional SQS/DLQ resources have no connected consumer, while Redpanda/Kafka and Kubernetes remain future decisions. Extraction should follow independent scaling, replay, ownership or deployment evidence. PyTorch, a second tree implementation and an extra animation engine are omitted without evidence of added value.

Implementation sources: README.md; docs/PRD.md

07 Complete system architecture

At context level, actors enter a decision-support system, while configured identity, licensed providers and an optional chain are external boundaries. The identity provider owns passwords and optional MFA. Providers remain untrusted until schema, provenance and licensing checks pass. Chain state can attest a digest but cannot establish that an observation was true.

At container level the browser enters through Next.js. The BFF resolves an opaque server session or explicit bearer identity, bounds input and calls Spring. Spring validates the token and current membership, applies tenant/owner scopes, accesses PostGIS/Redis and orchestrates Python. The Python service accepts protected internal credentials and a server-derived tenant header; deployment does not make it a public end-user API.

An explicit local adapter may evaluate synthetic/manual scenarios directly in Python when both preview flags are enabled. It does not retrieve tenant evidence, persist an audited Java scenario or prove production authorization. The interface labels that narrower computation as local and unsaved. Production must disable these flags.

The C4 and sequence diagrams in this report are generated from editable node/edge descriptions. They show current runtime and storage ownership rather than repeating obsolete phase-one diagrams. Planned infrastructure and operator actions are labeled separately; no asynchronous consumer, public deployment or distributed transaction is inferred from an arrow.

Implementation sources: apps/web/components/providers.tsx;
services/api/src/main/java/io/citypulse/decision/DecisionService.java

Figure 01 - C4 context: actors and external trust



CityPulse supports investigation by human actors. The identity provider and data source are separate trust boundaries. Source licensing and completeness are not established by authentication. Chain operations attest a content digest rather than establish measurement truth. The diagram does not assert a real identity-provider enrollment, licensed live feed, municipal partner or public-chain deployment.

Generated from editable source/diagrams.json and the inspected implementation.

Figure 02 - C4 containers: implemented runtime boundaries



Next mediates browser requests and Spring owns canonical authorization/persistence. Python isolates numerical libraries and uses trusted service tenant context. An explicitly enabled local synthetic/manual adapter is a narrower exception, not tenant retrieval or audited persistence. Blockchain submission is operator-driven in the local demonstration; no background attestation worker is implied.

Generated from editable source/diagrams.json and the inspected implementation.

08 Backend components and request lifecycle

Security filters establish identity, narrow permissions against enabled membership and limit requests before application services perform business work. Controllers validate DTOs and route to application operations. Registry repository ports separate domain-facing behavior from JDBC, while workspace operations use tenant-bound JPA/JDBC and transactional receipts. Error advice exposes meaningful known statuses without leaking raw exception details.

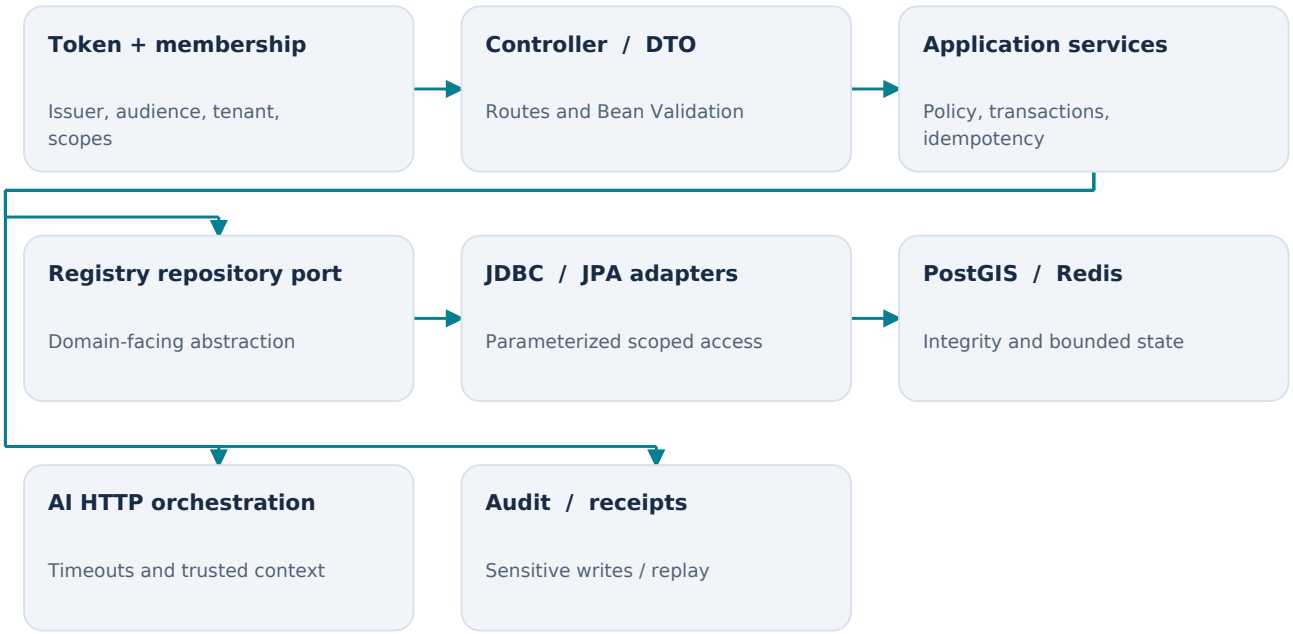
A decision request illustrates the separation. Next verifies origin, resolves authorization and bounds streamed bytes. Java derives tenant and actor, checks permissions, resolves an owned immutable snapshot and any licensed network reference, limits concurrent computation and calls Python. Python validates finite scenario dimensions and returns a solve status, diagnostics, canonical input, versions and fingerprints. Java persists the owned scenario/audit; the UI displays that actual response.

These layers provide fault isolation, not unlimited independence. Database or Redis readiness can affect API admission; Python failure affects numerical operations while registry reads may remain possible. A valid solver result does not grant permission to persist or dispatch it.

Idempotency guards retried writes; it does not turn an unavailable service into a successful action.

Implementation sources: apps/web/components/providers.tsx; services/api/src/main/java/io/citypulse/decision/DecisionService.java

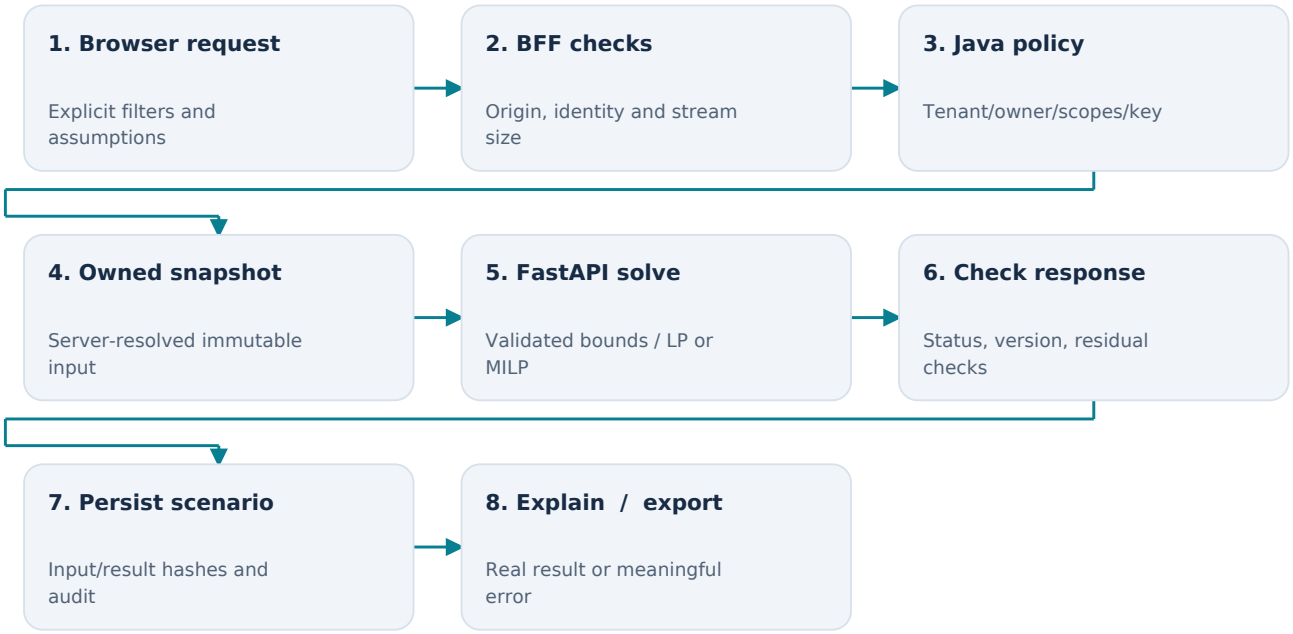
Figure 03 - C4 components: modular API



This view shows components within the modular Java boundary rather than independent business microservices. Permission checks are not delegated to UI visibility or model instructions. Repository ports, transactional services and scoped adapters keep business logic separate from controllers. Database owners remain administratively powerful even when runtime audit mutation is revoked.

Generated from editable source/diagrams.json and the inspected implementation.

Figure 04 - Sequence: authorized persisted decision



The numbered sequence makes the trust transitions explicit. A local unsaved scenario omits Java persistence and cannot be presented as an audited tenant result. Retried writes use scoped idempotency receipts. Solver infeasibility or unavailable upstream services are not replaced with sample plans. Export and reproduction refer to evaluated canonical inputs, not subsequent form edits.

Generated from editable source/diagrams.json and the inspected implementation.

09 Frontend architecture and accessible product behavior

The shared Next.js app includes marketing, features, sign-in, dashboard, explorer, intelligence, predictions, opportunities, optimization, verification, analyst, reports, notifications, settings, admin and help routes. Unknown sections return 404. TanStack Query owns request state and caching; Zustand persists local filters, theme, demo persona and read markers. These preferences never authorize server operations.

Recharts, MapLibre and deck.gl provide charts/overlays with textual or table alternatives. Heavy modules load selectively. Motion is restrained and disabled for reduced-motion preferences. Seed KPIs and illustrative charts remain labeled synthetic, while configured evidence, forecast jobs and optimization use real endpoints. A backend failure must not promote seed values into apparently live intelligence.

Issue #22 and PR #23 corrected portal theming: the selected theme now lives at the document root, the notification provider receives it explicitly, and appearance controls wait for saved-state hydration. The browser regression covers dark/light command dialogs, native controls, notifications, mobile navigation, keyboard focus and reload/navigation persistence. It also

exposed a 4.25:1 default success-toast contrast, corrected without weakening the accessibility check.

The interface is tested with automated axe checks and keyboard/mobile journeys. That is not a complete manual accessibility audit or screen-reader certification. The administrator persona changes navigation only. Authentication uses the configured OIDC provider and does not collect preview passwords. Additional real-user and assistive-technology evaluation remains part of pilot acceptance.

Implementation sources: apps/web/components/providers.tsx; services/api/src/main/java/io/citypulse/decision/DecisionService.java

10 Backend contracts, errors and implemented behavior

All protected operations derive tenant UUID from the signed tenant_id claim, or the fixed local tenant in explicit local profile. Membership must exist and be enabled. Effective permissions are the intersection of signed scopes and database role permissions. ADMIN can use all recognized scopes; ANALYST/PLANNER read registry/data and read/write analytics; PROVIDER/BUSINESS_OWNER read registry/data/analytics and write data; CITIZEN reads registry. Business-owner records are currently tenant-scoped, not individually owner-scoped. Do not enable mixed-owner tenants until row ownership policy is added.

OIDC resource server validates issuer, expiry and audience. Configure OIDC_ISSUER_URI and OIDC_AUDIENCE. The external provider owns passwords and login. No IdP realm is fabricated. Administrators provision membership against real IdP subjects; there is no self-service tenant creation or password API.

10.1 Resource groups under /api/v1

Path	Behavior	Required scope
GET /tenants/me; PATCH /tenants/me	Read/rename current tenant	registry:read / admin:write
GET/POST /organizations	Tenant list/create	registry:read / admin:write
GET/POST /users; PATCH /users/{id}/role	Membership list/provision/role and enabled state	admin:read / admin:write
GET/POST /cities	Bounded city list/create with geometry	registry:read / registry:write
GET /cities/{id}/zones; POST /zones	List/create city-contained zones	registry:read / registry:write
GET/POST /areas	Regions, districts, neighborhoods; parent containment	registry:read / registry:write
GET/POST /businesses	Search/sort/page and create located businesses	registry:read / registry:write
GET/POST /sources	Licensed provider catalog	data:read / data:write
GET/POST /observations	Historical query and validated append	data:read / data:write
GET/POST /inventories	Snapshot list and append	data:read / data:write
GET /analytics/imbalance	Sum compatible commodity/unit observations by zone	analytics:read
GET /opportunities	Positive-gap share score 0..100, heuristic	analytics:read
POST /optimizations	Proportional allocation scenario, never dispatch	analytics:write
GET/POST /forecasts	Stored results / real FastAPI orchestration	analytics:read / analytics:write
GET/POST /verifications	Stored digest receipts, status DIGEST_RECORDED	data:read / data:write
GET /notifications; PATCH /notifications/{id}/read	Recipient-scoped list/read	registry:read
POST /notifications	Create in current tenant	admin:write
GET /admin/audit	Paginated structured audit	admin:read
GET/PUT /admin/configuration	Display name / timezone only, no secrets	admin:read / admin:write
GET /reports/export	Tenant CSV, first 100 observations, explicit X-Export-Limit	data:read
POST /imports/observations	application/json multipart, 1 MB, 1..500 rows, atomic	data:write

POST business operations require Idempotency-Key of 8..100 safe characters. Receipts are scoped by tenant/subject/operation/key and digest. Concurrent requests serialize on a

transaction advisory lock. Identical retries return the stored result; mismatched requests return 409. Receipt expiry/deletion is not yet scheduled.

List endpoints use page 0..100000 and size 1..100 where exposed. Zone lists and aggregate results cap at 100. Business sort allows only name or createdAt; search is parameterized. Observation and aggregate queries require cityId, commodity, unit, from and to, with a positive window no longer than 366 days. Only compatible units/commodities are summed; incomplete coverage must not be mistaken for complete demand.

Redis enforces 180 authenticated API requests per subject/tenant per 60-second fixed window, with remaining/limit headers and Retry-After. Failure to verify rate limits fails closed. City reads cache for 30 s with tenant-aware keys; new cities may appear after TTL. OIDC tokens never gain permissions from a database role alone. CORS defaults to same-origin. CSRF remains except for explicit Bearer API requests. The browser supports configured OIDC/PKCE server sessions; actual provider enrollment remains unverified.

Geometry, references, quantities, times, schema/JSON constraints and Bean Validation reject invalid writes. Sensitive writes audit in the same transaction. No SQL is assembled from caller-supplied identifiers except allowlisted sort constants. PII is minimized to subject/display name. Retention policy is documented but automated deletion/legal holds remain incomplete.

A DIGEST_RECORDED entry is not an on-chain attestation. Receipt confirmation is separately implemented as described below. Licensed live connectors, asynchronous exports, tenant onboarding/deletion and a complete update/archive lifecycle remain incomplete.

10.2 Advanced forecast integration

POST /api/v1/forecast-jobs requires analytics:write and Idempotency-Key. Body: city/district/category slugs, ordered daily observations {time,demand,supply?}, horizon 1..90 and explicit dataMode. Java derives the tenant, calls FastAPI /v1/forecasts with service identity and X-Tenant-ID, and persists the returned job. GET /api/v1/forecast-jobs/{id} requires analytics:read and retrieves only the current tenant's result. The dashboard forecast lab calls this real path and displays returned intervals or an explicit unavailable state. See the model card.

GET /api/v1/analytics/metrics adds mode/duration-separated ratio, shortage/surplus, UTC time buckets, volatility, business-category distribution, geodesic business density, local-UTM clustering and an explicitly limited proximity indicator. No missing population or capacity inputs are fabricated.

10.3 Blockchain receipt confirmation

POST /api/v1/verifications/{id}/receipt requires data:write and a current-tenant digest record. It validates the transaction against configured RPC, chain ID, registry address, attestation-event topic, attestation ID, content digest and canonical block hash before saving CHAIN_CONFIRMED. Unconfigured/unreachable RPC returns 503; unmatched or unmined evidence returns 409. A successful verification request without that matching receipt remains DIGEST_RECORDED. See blockchain guide for actual local demo evidence, role design and finality limitations.

10.4 PulseAI

The /api/v1/pulseai namespace provides owned sessions and evidence-grounded messages. See the PulseAI contract and security guide for requests, limits, retrieval semantics and current limitations. Runtime OpenAPI includes validated question fields and the PulseAiAnswer response schema.

Implementation sources: docs/api/backend.md

11 Database, PostGIS and entity relationships

The schema evolves through versioned migrations: city/zone/business/source/observation/inventory/audit are extended by tenants, organizations, memberships, geographic hierarchy, notifications, operation receipts, forecasts, chain confirmation, conversations and decision objects. V1 alone is not the complete protected schema. The appendix extracts physical declarations and later column additions from the actual SQL.

Cities/zones use nonempty valid WGS84 MultiPolygons. Business locations are longitude-first Points. Geographic-parent triggers and composite references constrain inserts/updates. Parent boundary edits still require controlled descendant revalidation. GiST supports spatial access; tenant/time and tenant/name indexes support protected operational queries. Distances and areas use geography or explicit projected coordinates rather than raw degrees interpreted as metres.

City definitions can be shared through tenant_city assignment, so the ERD does not falsely make every city directly tenant-owned. Business/source/observation/inventory rows carry tenant scope, with composite constraints for sensitive references. Conversations and decision objects add subject ownership. General business rows are not individually owner-scoped inside a tenant, which gates mixed-owner production use.

Observation quantity, commodity, unit and event interval define a flow. Inventory is a recorded stock snapshot. ingested_at is distinct from observed_at. Runtime grants permit required reads/inserts but not schema changes, migration-history access or audit mutation; owner credentials remain powerful. Append-only application grants are not an external immutable log service.

Implementation sources: services/api/src/main/resources/db/migration/V1__canonical_geospatial_schema.sql; services/api/src/main/resources/db/migration/V2__tenant_operations.sql

Figure 05 - ERD: principal logical relationships



The physical appendix is definitive for column declarations and composite constraints. City is assigned through tenant_city rather than falsely shown as directly tenant-owned. Business/source/observation and inventory references include tenant protection. Chat and decision objects narrow access by subject ownership. This overview omits some foreign keys for readability and is not a full database introspection.

Generated from editable source/diagrams.json and the inspected implementation.

12 Data sources, licensing and retention semantics

EPSG:4326 is canonical, longitude before latitude. Cities/zones use valid nonempty MultiPolygons; business Points must lie within their zone. GiST indexes support spatial queries. Inserts and child updates enforce containment. A future parent-boundary write service must lock and revalidate descendants; arbitrary parent geometry SQL edits are unsupported. Cast to geography for geodesic meters, never treat degrees as meters.

UUIDs identify records. UTC timestamptz separates event and ingestion time; IANA timezone names control display. Future ingestion validates timezones against the runtime database, coordinate bounds, polygon validity, unit compatibility, provider licensing, and temporal windows.

Supply/demand share an observation table distinguished by kind. Periods must be positive, quantities nonnegative, quality scores 0..1. Never aggregate incompatible units, commodities, or windows. Inventories are snapshots rather than transactions. Every observation requires source metadata, payload SHA-256, and object provenance. (source_id, external_id) is unique; future ingestion returns 409 if that key is reused with a different digest. Hash a documented canonical representation for real attestation. Fixture hashes explicitly hash a synthetic source string.

Validation sequence: bound file size/type; validate interchange schema and provider scope; resolve geometry and business/city consistency; deduplicate and detect conflicts; write observation/audit atomically; quarantine rejected rows with sanitized diagnostics. Schema constraints exist; upload and quarantine services are planned.

12.1 Proposed retention

Data	Duration	Action
Raw uploads and rejects	30 days	Delete after import/investigation
Detailed observations/inventories	24 months	Aggregate then delete under source agreement
Anonymous monthly aggregates	5 years	Annual reidentification review
Audit events	12 months	Restricted access; legal holds override
Local browser preferences	Until cleared	User controlled

These are policy targets requiring jurisdiction/provider review. No automated deletion runs. Retention preview is read-only. Implement legal holds, deletion workers, backup expiry and monitoring before a pilot. Chain stores digests only; deleting off-chain data cannot erase a chain digest.

Implementation sources: docs/data-policy.md

13 Ingestion, quality and idempotent publication

13.1 Reproducible local demo

```
python scripts/data_pipeline.py demo --city bengaluru --seed 42
python scripts/data_pipeline.py demo --city amsterdam --seed 42 --out .data/amsterdam
python -m unittest discover -s services/ingestion/tests -v
```

These create 28 days of synthetic supply/demand, CSV/JSON/Point-GeoJSON variants, catalog metadata, normalized records, quality report, dead letters and metrics. Seed, profile and UTC epoch are fixed. Ingestion timestamps record actual execution separately. Example boxes are not administrative boundaries. INR/EUR are explicit catalog currencies; no price model or currency conversion is implied.

13.2 Pipeline and refresh

```
python scripts/data_pipeline.py ingest --input data/sample/generated-bengaluru/observations.csv --catalog
data/sample/generated-bengaluru/catalog.json --out .data/csv-import
python scripts/data_pipeline.py ingest --input data/sample/generated-bengaluru/observations.geojson --catalog
data/sample/generated-bengaluru/catalog.json --out .data/geo-import
# With the API stack running and .env configured:
python scripts/data_pipeline.py publish
```

Stages are bounded raw read, required/schema checks, whitespace/category cleanup, finite decimal validation, offset-to-UTC normalization, IANA timezone/currency checks, WGS84 bounds and configured rectangle checks, source/external-ID digest deduplication, explicit quality scoring, grouped temporal metrics, and API publication into PostGIS-indexed storage. Polygon containment on publication is enforced by the registry. Quality is a transparent rule score (1 with validated coordinates, 0.8 with IDs only), not statistical confidence or source truth.

Unknown columns are rejected to avoid accidental personal-data collection. No individual location traces are generated. Dead letters contain row numbers and reasons, not raw private input. Duplicate identical records are skipped; changed duplicate digests are quarantined. Local checkpoints support incremental refresh. Use one process per checkpoint directory; distributed execution is not implemented. File-level schema errors fail the job before checkpointing.

Publication creates/replays a city, zone and synthetic source through real scoped APIs, then writes observations with deterministic idempotency keys. It retries only temporary HTTP/network failures up to three attempts with bounded delays. Failed publication writes a replayable dead-letter diagnostic and never records a false success. Run publish again after restoring connectivity. Database receipts ensure successful writes are replayed. Local state is not an exactly-once distributed system.

13.3 Public API adapter

```
python scripts/fetch_open_data.py data/official-census-source.json
```

The reviewed example uses official TIGERweb States and Census federal-work policy. It snapshots a bounded 2026-vintage Delaware boundary, not city demand. Raw licensed geography requires a reviewed domain mapping before publishing as a city/area. It never becomes demand observations automatically. No private or real-time feed is claimed.

The fetcher requires an exact reviewed HTTPS host, explicit verified license reference, no URL credentials, no redirects, JSON validation and a 10 MB cap. ETag enables unchanged-snapshot refresh when supported. Network failure is reported. It stores provider metadata, license, fetch time and payload digest separately from raw data.

Amsterdam areas documentation marks the dataset public but leaves its license field unspecified; the catalog therefore blocks it pending license review. India OGD describes API-key access; none is configured. The Government Open Data License India does not imply access to every endpoint. No keys or access are fabricated.

13.4 Metrics and publication

Offline metrics partition commodity, unit, mode and interval duration before sums. They return demand/supply ratio (null for zero supply), shortage/surplus, gap share, observed-interval standard deviation and coefficient of variation, time series, and same-time previous-week comparisons. Missing rows are unknown, not verified zero. Week-over-week differences do not establish seasonality. Resource utilization is returned only when compatible supply capacity is present; otherwise null.

Observation locations can form connected components within 1 km using haversine distance (not DBSCAN or population clusters). Local computation caps unique locations at 2000 and reports when skipped. Real registered businesses use PostGIS DBSCAN with local UTM, business density per geodesic square kilometer, and a business-neighbor-within-1km proxy. This proxy is not resident accessibility, walking time, or transport access. City-scale clustering assumes one UTM zone and excludes polar regions.

GET /api/v1/analytics/metrics publishes tenant-scoped stored metrics with cityId, commodity, unit, from, to. It separates source mode and interval duration and includes UTC daily buckets, volatility, category-level business density, geographic clusters and proxy disclosures. Supply/demand imbalance and scoring endpoints remain documented separately. The intelligence UI has a real local-evidence panel alongside explicitly synthetic charts.

Population-weighted accessibility, real capacity utilization, validated seasonality, travel-network routing, source coverage assurance and public-feed schedules require licensed datasets and remain unsupported where inputs are absent. No substitute numbers are invented.

Implementation sources: docs/ingestion.md

Figure 06 - Data ingestion and publication lifecycle



Local normalization and remote publication have different success boundaries. A written checkpoint is not evidence that every API write arrived. Repeated publication relies on scoped server receipts. Quality is a rule-based diagnostic, not source truth or model confidence. Explicit modes and compatible time/unit/geographic definitions must survive each transformation.

Generated from editable source/diagrams.json and the inspected implementation.

14 Supply-demand analytical methodology

For compatible records define demand D and supply S. Imbalance is D - S; shortage is max(D - S, 0); surplus is max(S - D, 0). Demand/supply ratio is undefined for zero supply, while gap share requires positive demand. These quantities describe the selected records, not complete citywide unmet need.

Compatibility is the substantive assumption: commodity, unit, geography, interval, mode and coverage must agree. AnalyticsService separates source modes and interval durations, uses UTC daily buckets and discloses limits. Some legacy conditional sums return zero when a side is absent; shortage/ratio fields therefore require missing-coverage caution. Newer PulseAI groups and decision snapshots preserve an absent counterpart as unknown. Neither proves complete source coverage without an upstream coverage contract.

Business density uses registered counts divided by geodesic city area. The proximity indicator counts businesses with another registered business within one kilometre; it is not population accessibility. DBSCAN projects small-city points to local UTM, using a one-kilometre neighbourhood and two-point minimum. Large multi-zone geographies, high latitudes, incomplete registrations and network accessibility require other methods.

Historical comparisons need matching cadence and local-calendar interpretation. UTC day buckets can cross the city's calendar boundary. Volatility and seasonal differences are indicators, not causal explanations. The system does not infer population, capacity, opening hours, road travel time or resource utilization from absent datasets.

Implementation sources: services/api/src/main/java/io/citypulse/workspace/application/AnalyticsService.java

15 AI forecasting methods and feature engineering

15.1 Purpose and data

Local decision-support experiments for daily demand by city, district and category. The reproducible benchmark uses eight synthetic segments across Bengaluru/Amsterdam, central/north and groceries/mobility. These city labels do not establish real city observations. No measured synthetic error implies real-world accuracy.

15.2 Training and evaluation

scripts/train_models.py fixes random seed 42 and generates 182 daily observations per segment. Models train separately per geographic/category segment. A chronological 60/20/20 split prevents future targets entering training. Lag 1/7/14 and the previous-seven mean use only past observations; weekday sine/cosine are known calendar features. No pooling across geographies is used.

Candidates: seasonal-naive, seven-point moving average, Ridge regression and XGBoost. Validation MAE selects the model independently per segment. The deployed artifact remains trained on the training split only. Held-out test results and all candidate comparisons are computed in evaluation JSON. Metrics include MAE, RMSE, sMAPE and mean under/overforecast. These are rolling one-step evaluations, where prior observed test values are available at each subsequent origin; they are not claims about long-horizon recursive accuracy.

PyTorch and a second tree implementation are omitted because the benchmark provides no evidence that additional complexity would improve decisions.

15.3 Intervals and explanations

Intervals use the selected model's 90th percentile absolute validation residual. Validation also selects the model, so this is an empirical diagnostic, not an independent conformal guarantee. The test report measures one-step interval coverage. Recursive horizons widen that radius by square root of horizon, a heuristic with no guaranteed multi-horizon coverage. Without a trained calibration artifact, intervals are null and clearly disclosed.

XGBoost split importance and Ridge raw coefficients are associations, not causal explanations or comparable normalized weights. Baselines explain the repeated weekday or previous-seven average. Anomalies use past-only median/MAD. Drift compares recent seven-day mean with training mean/standard deviation; trends and seasonality can trigger it, so it is diagnostic rather than a validated alarm.

15.4 Service behavior

POST /v1/forecasts, GET /v1/forecasts/{id}, POST /v1/forecasts/batch, GET /v1/models, GET /v1/models/{id}/metrics, and /health run in FastAPI. Forecast jobs persist in SQLite scoped by a trusted service X-Tenant-ID header. Service bearer authentication is mandatory. Spring derives that tenant from authenticated identity, applies permissions and idempotency, orchestrates inference and persists the result. The browser never chooses a trusted tenant header.

Daily requests require at least 14 ordered offset-aware observations, bounded values, a known scope and horizon 1..90. Batch is bounded to 20 series and 10k total observations. A body limit precedes validation. Missing or unreadable models fall back to seasonal-naive without fabricated intervals. Synthetic models are not used automatically for nonsynthetic requests. Supply forecasts repeat last week's known supply; gap flags and additional-unit recommendations are simple scenarios requiring human review. Nothing dispatches resources.

15.5 Registry, reproducibility and rollback

Training fingerprints sorted dataset records and names model versions from the digest. Artifacts are inspectable Ridge JSON or XGBoost JSON with no untrusted pickle loading. Only trusted local registry files are loaded. MODEL_REGISTRY_DIR selects the registry; AI_STORAGE_DIR controls persistent jobs. Cached models keep inference separate from training.

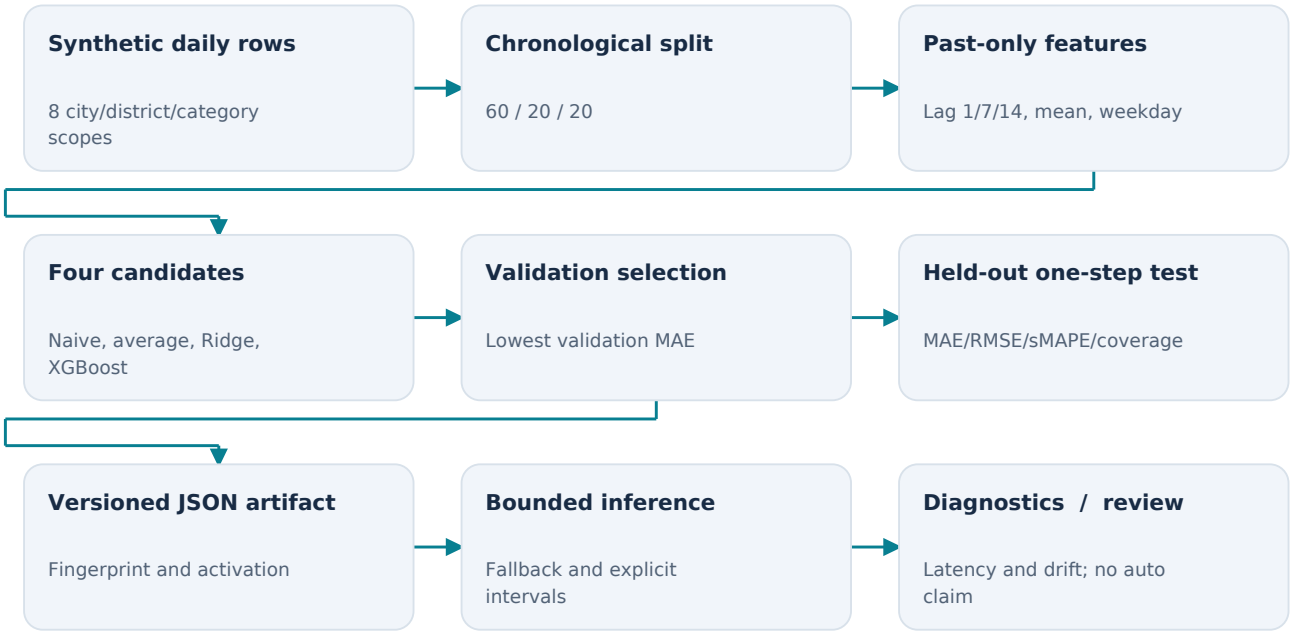
Run python scripts/model_registry.py list, then python scripts/model_registry.py activate --version <existing-version> to promote or roll back. The previous pointer is preserved. The weekly synthetic evaluation workflow trains and uploads artifacts; it does not auto-promote a production model. Real-data training requires provider licensing, dataset review, segment-specific backtesting, coverage review and explicit promotion policy.

15.6 Limits

Daily cadence only; no irregular-series resampling, holiday data, prices, exogenous weather, geospatial transfer, LLM recommendations or population access model. SQLite is a local persistence choice; shared production deployment needs durable central storage and service identity. Forecast retention, tracing and production SLOs remain operational follow-up. Prediction latency is measured per response; benchmark it under representative load before setting a guarantee.

Implementation sources: docs/model-card.md

Figure 07 - ML lifecycle: chronological segmented evaluation



Each segment is separate; the experiment does not establish unseen-city transfer. Rolling one-step tests use earlier observed targets at later origins. Validation selects the model and also supplies residual intervals, so interval coverage is diagnostic rather than an independent conformal guarantee. Recursive horizon widening requires separate evaluation.

Generated from editable source/diagrams.json and the inspected implementation.

16 Training protocol and actual evaluation results

The committed evaluation includes eight synthetic city/district/category segments with 182 daily rows each. Chronological 60/20/20 splits separate parameter fitting, validation selection and held-out reporting. Features are lag 1/7/14, the previous-seven mean and known weekday sine/cosine. Geographic/category models are separate; no unseen-city transfer result is claimed.

Candidates are seasonal-naive, seven-point moving average, Ridge with alpha 10 and bounded XGBoost. Validation MAE selects a candidate per segment. The complete candidate table is retained, including cases where the selected candidate is not best on the untouched test period. Charts read the committed JSON, not invented scores.

MAE measures absolute error in the synthetic target units; RMSE emphasizes larger errors; sMAPE uses the implemented symmetric denominator with zero contribution when both values are zero. Directional under/overforecast exposes asymmetric risk. Evaluation is rolling one-step: prior observed test values become available at later origins, while no future target enters its own features. This does not establish recursive multi-horizon accuracy.

Intervals use a 90th percentile absolute validation residual, but validation also selects the model. They are empirical diagnostics, not independently calibrated conformal guarantees. Square-root

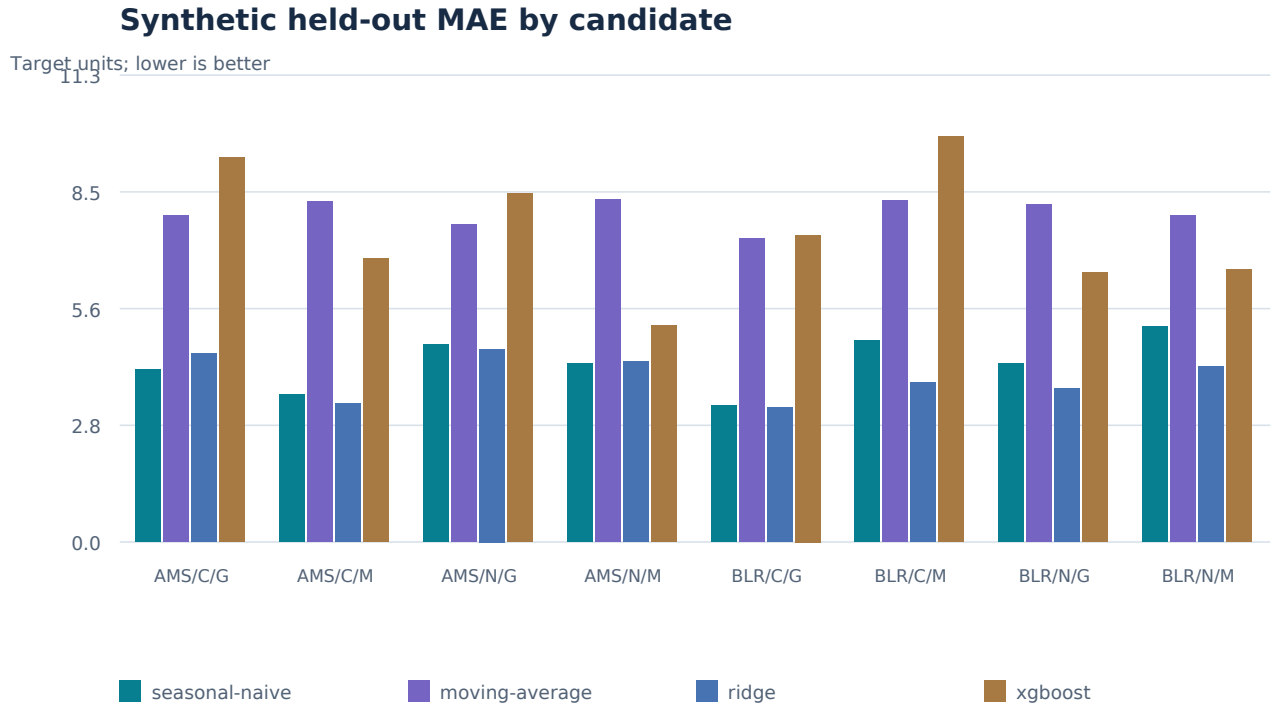
horizon widening is heuristic. Measured synthetic coverage must not be generalized to a real provider, changing reporting process or unseen geography. Real adoption requires separate calibration and multi-horizon backtests.

Implementation sources: docs/reports/model-evaluation.json

Measured interpretation: selected-model test MAE ranges from 3.255 to 5.196 synthetic target units. One-step empirical interval coverage ranges from 62.2% to 100.0%, which is not uniformly nominal 90% coverage. In 3 of 8 segments the validation-selected candidate is not the lowest-test-MAE candidate. This is evidence for transparent model comparison and separate calibration, not a real-city accuracy claim.

Selected candidates: held-out one-step results

Synthetic segment	Selected	MAE	RMSE	sMAPE %	Coverage
amsterdam/central/groceries	ridge	4.546	5.710	3.38	0.946
amsterdam/central/mobility	ridge	3.346	4.306	2.49	0.892
amsterdam/north/groceries	ridge	4.658	5.424	3.46	0.703
amsterdam/north/mobility	ridge	4.351	5.728	3.29	0.784
bengaluru/central/groceries	ridge	3.255	4.103	2.46	1.000
bengaluru/central/mobility	ridge	3.842	4.881	2.90	0.946
bengaluru/north/groceries	ridge	3.705	4.424	2.75	1.000
bengaluru/north/mobility	seasonal-naive	5.196	6.234	3.89	0.622



AMS/BLR label synthetic segments; C/N are central/north and G/M groceries/mobility. Compare within segment; this is not measured real-city accuracy.

All candidate comparisons

Synthetic segment	Candidate	Validation MAE	Test MAE	Test n
amsterdam/central/groceries	seasonal-naive	5.484	4.172	37
amsterdam/central/groceries	moving-average	8.918	7.884	37
amsterdam/central/groceries	ridge	4.457	4.546	37
amsterdam/central/groceries	xgboost	6.895	9.282	37
amsterdam/central/mobility	seasonal-naive	3.500	3.571	37
amsterdam/central/mobility	moving-average	9.061	8.222	37
amsterdam/central/mobility	ridge	3.267	3.346	37
amsterdam/central/mobility	xgboost	4.855	6.847	37
amsterdam/north/groceries	seasonal-naive	3.690	4.760	37
amsterdam/north/groceries	moving-average	8.277	7.662	37
amsterdam/north/groceries	ridge	3.601	4.658	37
amsterdam/north/groceries	xgboost	4.349	8.402	37
amsterdam/north/mobility	seasonal-naive	3.837	4.300	37
amsterdam/north/mobility	moving-average	7.637	8.261	37
amsterdam/north/mobility	ridge	2.947	4.351	37
amsterdam/north/mobility	xgboost	3.917	5.227	37
bengaluru/central/groceries	seasonal-naive	4.849	3.303	37
bengaluru/central/groceries	moving-average	8.796	7.316	37
bengaluru/central/groceries	ridge	4.136	3.255	37
bengaluru/central/groceries	xgboost	5.635	7.409	37
bengaluru/central/mobility	seasonal-naive	5.391	4.869	37
bengaluru/central/mobility	moving-average	9.212	8.235	37
bengaluru/central/mobility	ridge	4.259	3.842	37
bengaluru/central/mobility	xgboost	5.695	9.797	37
bengaluru/north/groceries	seasonal-naive	4.969	4.318	37
bengaluru/north/groceries	moving-average	7.657	8.150	37
bengaluru/north/groceries	ridge	4.117	3.705	37
bengaluru/north/groceries	xgboost	6.151	6.510	37
bengaluru/north/mobility	seasonal-naive	3.037	5.196	37
bengaluru/north/mobility	moving-average	7.749	7.873	37
bengaluru/north/mobility	ridge	3.233	4.240	37

Synthetic segment	Candidate	Validation MAE	Test MAE	Test n
bengaluru/north/mobility	xgboost	4.038	6.577	37

Dataset fingerprint: 49ffebe53361be13ce46d58eb1a819a86b1fb9ba1936bc149224e74d7d3ddbff

17 Blockchain design and proof verification

This implementation is a local prototype, not an audited contract system. It stores public digests and hashed references only. It never stores raw private observations or personal data on chain.

17.1 Modules

Contract	Responsibility
AuthorizedSourceRegistry	Registrar binds a wallet to an organization digest and metadata digest, and can revoke future authorization
DatasetAttestationRegistry	Authorized attester records event/content/reference hashes and a use-case kind; emergency pauser can stop writes
VerificationRecord	Read-only comparison of a known attestation against an independently calculated content digest

OpenZeppelin AccessControlDefaultAdminRules uses a delayed two-step default-admin transfer. Registrar, attester and pauser roles remain distinct. Production administration should use a separately secured multisig. Roles and registration establish administrative authority, not the truth of a real-world identity or dataset.

Kinds: 1 dataset, 2 supply observation, 3 authorized audit event, 4 approved allocation, 5 report. A kind-4 record is an attester's assertion of an off-chain approval; the contract does not independently run a municipal approval workflow. A separate approval registry is not added without that workflow.

Duplicate event IDs are rejected per submitter. Attestation IDs include chain ID, contract address, sender and event ID, avoiding cross-deployment ambiguity. A repeated content hash may be legitimate for a new publication/event, so duplicate protection operates on event identity rather than prohibiting all content reuse. No meta-transactions or arbitrary signatures are accepted, reducing replay surface.

17.2 Local demonstration

```
npm ci
# Terminal 1: funded development accounts on loopback only
npm run node --workspace blockchain
# Terminal 2: deploy, attest a synthetic CSV, read receipt and verify/tamper-check
npm run demo:local --workspace blockchain
npm run verify:report --workspace blockchain
```

For a self-contained ephemeral chain without an RPC server, run npm run demo:ephemeral --workspace blockchain. This deploys, transacts, checks the real receipt, and reads the

independent verifier during the same run; the chain ends with the process and cannot be queried later. Captured local evidence records an actual ephemeral run. It is not a public deployment.

Persistent local demo evidence also records a separate-process report verification on local port 8547. Set LOCAL_CHAIN_RPC_URL to select a nondefault loopback RPC when port 8545 is in use.

The demo restricts itself to chain 31337 and creates a report in .data/blockchain/sample-report.csv. It estimates gas before sending, waits for the actual receipt, records gas used, transaction/block hashes, addresses, chain ID and contract version, and independently reads the verification contract. Tampered bytes must return false. Keys printed by Hardhat are disposable development keys and must never secure public assets. Chain restart invalidates old local deployments; the independent verifier checks network and deployed bytecode.

Exact-byte SHA-256 is used for file content. UTF-8 strings must preserve newlines and whitespace. Off-chain reference strings are SHA-256 hashed before submission, avoiding accidental publication of signed URLs or private paths. Keep a controlled off-chain mapping from reference digest to storage location. A digest of predictable personal data can still leak information through guessing; do not attest personal records without a reviewed privacy design.

17.3 Backend and UI

Configure the API from the real local result manifest:

```
BLOCKCHAIN_RPC_URL=http://localhost:8545
BLOCKCHAIN_CHAIN_ID=31337
ATTESTATION_CONTRACT_ADDRESS=<DatasetAttestationRegistry address from the demo>
ATTESTATION_EVENT_TOPIC=<eventTopic from the demo>
ATTESTATION_CONTRACT_VERSION=1.0.0
```

The Java backend never needs a private key. Create a digest receipt using POST /api/v1/verifications with {payload:<exact report text>} and an Idempotency-Key. Submit {transactionHash,attestationId} to POST /api/v1/verifications/{id}/receipt. Server-side scope and tenant checks precede RPC verification. The server verifies the configured chain ID, successful receipt, registry address, event signature, attestation ID, digest topic and canonical block hash before persisting CHAIN_CONFIRMED. Unconfigured RPC returns 503; pending, failed or mismatched evidence returns 409.

Set CITYPULSE_API_URL and LOCAL_API_TOKEN in the demo shell to perform those two backend steps automatically after the actual chain transaction. The CLI reports backend failure independently; it never erases or falsely confirms a chain transaction. The verification center and administrator console show backend receipt records, or an explicit unavailable/empty state. Local networks get no fabricated explorer link.

For Compose API access to a host Hardhat process, configure a reachable private host address and firewall deliberately; the provided node stays loopback-only. A host-side API is the simplest local demo. A Docker chain profile is not deployed automatically.

17.4 Optional testnet deployment

Use npm run deploy:testnet --workspace blockchain only after setting TESTNET_RPC_URL, TESTNET_PRIVATE_KEY and CONTRACT_ADMIN_ADDRESS in the shell or an ignored environment source. The network config uses Hardhat configuration variables; it does not read committed keys. Never reuse local keys. No public deployment is performed by this task and no explorer URL is invented. Configure an explorer only after verifying chain ID, transaction hash and deployment address. Use faucet test funds only and understand gas before sending.

17.5 Threat model

Threat	Control and limit
Unauthorized writer	OpenZeppelin roles plus active source registration; compromised authorized keys remain a risk
Role takeover	Delayed two-step default admin transfer; use multisig and separate operational keys
Duplicate/replayed event	Per-sender event ID uniqueness and chain/contract domain separation
Invalid proof/tampered file	Exact digest comparison, nonzero values, deployment and event matching
RPC fabrication	Trusted configured RPC required; public deployments need independent providers and stronger finality policy
Reorganization	Canonical block checked at submission; later revalidation/finality tracking remains a production requirement
Data deletion	Immutable digests/events cannot be erased; delete off-chain content under policy and disclose permanent commitments
Metadata leakage	Hash references, prohibit secrets/PII; hashes may still reveal low-entropy inputs
Gas griefing	Fixed-size hash inputs, no unbounded on-chain loops or external value transfers
Source revocation	Blocks future attestations without rewriting history; old attestations remain verifiable

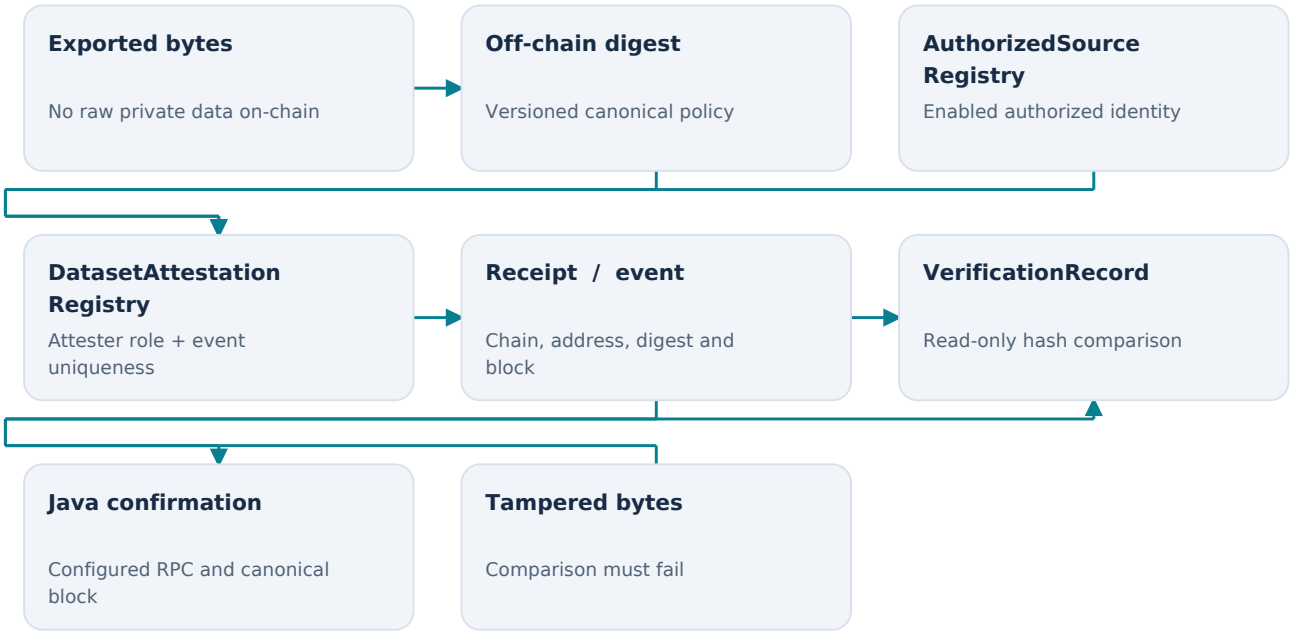
No contract holds currency, calls untrusted external addresses with value, uses upgrade proxies, or executes arbitrary payloads. The immutable source-registry dependency is trusted at deployment. Static analysis and tests are partial evidence, not an audit. Public operation requires independent review, signer rotation, multisig runbooks, finality monitoring and incident exercises.

17.6 Static checks

npm run lint --workspace blockchain runs Solhint security/style rules. The latest local run reported zero errors and six warnings (gas packing/indexing and workspace import-path resolution). Compilation resolves OpenZeppelin imports and contract tests execute them. Solhint is a limited static check, not a formal proof or an audit. No Slither audit result is claimed.

Implementation sources: docs/blockchain.md

Figure 08 - Blockchain proof verification



Attestation IDs include chain ID, registry address, submitter and event ID. The local sample reports a matched original digest and a failed tampered comparison. A DIGEST_RECORDED database row is not CHAIN_CONFIRMED until receipt checks succeed. Reorg/finality, signer custody, source quality and deletion limits remain operational responsibilities.

Generated from editable source/diagrams.json and the inspected implementation.

18 PulseAI analyst, retrieval and safety controls

PulseAI is a read-only analyst at /analyst. It retrieves actual tenant-authorized database records. The dashboard's seeded district figures never substitute for missing records. The default local fixture is synthetic, with rice demand and no matching supply; PulseAI explicitly reports that a shortage cannot be established.

18.1 Run

```
python scripts/setup-env.py
docker compose up --build -d
python scripts/smoke.py
docker compose down
```

Open http://localhost:3000/analyst. Compose explicitly enables a shared local development identity. Do not expose this configuration publicly. Outside Compose, set API_BASE_URL and PULSEAI_LOCAL_PREVIEW=true only for local development. The adapter otherwise requires the caller's Bearer access token; it never trusts a browser-supplied tenant or role. The browser

supports configured OIDC/PKCE server sessions. External-provider enrollment and MFA rollout remain unverified; direct clients use the resource-server boundary.

18.2 Contract

All routes require analytics:read **and** data:read, narrowed by active database membership on every request, including history. Tenant comes from the validated identity. The city picker additionally requires registry:read. A session belongs to the subject that created it.

Method	Route under /api/v1/pulseai	Behavior
GET	/cities	Up to 100 tenant-assigned cities, names, country codes and time zones
POST	/sessions	Create an owned seven-day session; maximum 30 active sessions
GET	/sessions	List up to 30 owned sessions
GET	/sessions/{id}	Up to 20 stored question/answer pairs, oldest first
DELETE	/sessions/{id}	Delete owned session and cascade messages
POST	/sessions/{id}/messages	Validate filters, retrieve evidence, select statements, persist answer

Use **Data context & filters to Load permitted cities** to choose an assigned city by name and time zone. The collapsed context keeps chat actions visible on mobile.

Message body:

```
{
"question": "Explain source provenance and missing supply",
"cityId": "10000000-0000-0000-0000-000000000001",
"from": "2026-01-01T00:00:00Z",
"to": "2026-01-02T00:00:00Z",
"commodity": "rice",
"unit": "kg",
"forecastId": null,
"comparisonCommodity": null
}
```

A second optional commodity can be compared only in the same measurement unit. Positive gaps are investigation candidates, never investment recommendations.

The interval is positive, at most 366 days and half-open. Only observations fully contained in it are considered. Units are count, kg, litre, kwh, or trip. Question length is 2,000 characters. Session writes lock the owned session to enforce the 20-message bound. Unknown/foreign/expired session or city returns 404; revoked scope returns 403. Bean Validation returns the existing problem-details schema. Ten API requests/minute per subject/tenant apply to the PulseAI namespace; Redis failure rejects requests. Existing global abuse controls remain in place elsewhere.

Each answer includes statements with actual observation UUIDs or source/forecast UUIDs, documentation paths and SHA-256 fingerprints, limitations, related views, follow-up prompts, and a typed line-chart/filter configuration that preserves source-mode and interval-duration separation. It never applies configuration or changes a dashboard automatically. Record references are identifiers, not public URLs. The authorized forecast endpoint provides the selected forecast in full.

18.3 Retrieval and truth boundary

• Spring executes fixed, parameterized, tenant-filtered read-only queries. There

is no model SQL execution, arbitrary tool name, network retrieval, or file path.

• Quantities group by zone, exact observation interval, source mode and unit.

Demand and supply must both be present before a gap is calculated. Inventory stocks are never treated as supply flows. Multiple sources are summed as reported; cross-source coverage and deduplication must be assessed upstream.

• At most 12 latest interval groups and six source references are returned. This

bounded evidence window is not a citywide ranking or a representative sample.

• Optional forecast UUID retrieval is tenant-checked. Forecast segment labels and

interval method come from the stored result. A slug-labelled forecast is not silently attributed to a registry city UUID.

• Documentation retrieval searches a build-time allowlist of docs/model-card.md

and docs/data-policy.md snapshots. It returns bounded plain-text excerpts and content hashes, never interprets embedded instructions, and follows no links.

• Optional LLM output is only a JSON list of allowed statement IDs. Both Python

and Java validate the IDs. Factual prose and citations are constructed by Java; provider-generated claims, instructions and citations cannot enter the answer. Documentation and source text are untrusted and rendered as escaped text.

The assistant can compare observed intervals and zones for up to two explicit commodities and one unit, explain source metadata and stored forecast methodology, and expose related geography/forecast views. Citywide cross-category rankings, inferred expansion hotspots, causal explanations, fairness-aware allocations and autonomous dashboard changes are **not implemented**. It declines conclusions unsupported by the selected records. It does not infer filters from natural language; use the context controls. Follow-up prompts re-run the same protected retrieval with the current filters.

18.4 Optional model and costs

The default PULSEAI_PROVIDER=disabled makes zero external model calls. FastAPI failure also falls back to verified statements. To enable a compatible provider:

```
PULSEAI_PROVIDER=openai-compatible
PULSEAI_PROVIDER_URL=https://api.openai.com/v1
PULSEAI_MODEL=<operator-selected compatible model>
PULSEAI_API_KEY=<secret supplied at runtime>
PULSEAI_MAX_USD_PER_MILLION_TOKENS=<ceiling covering BOTH input and output>
```

The adapter uses Chat Completions structured JSON output, store:false, an output cap of 256 tokens, HTTPS, an eight-second timeout, no redirects and no retry. Model/provider compatibility must be tested by the operator; no paid credentials were used for the reproducible tests. See the official structured-output contract.

SQLite atomically reserves at most 20 external attempts and 50,000 conservatively estimated tokens per subject/tenant/UTC day. Each reservation uses the UTF-8 payload byte count plus 2,000 tokens for schema, instructions and output. Requests above 16,000 reserved tokens fall back. Failed/ambiguous calls retain their reservation. The configured price ceiling must be positive and at most USD 20 per million tokens: with valid operator-supplied prices the token ceiling corresponds to at most USD 1/day/subject, excluding provider-specific fees. This is an application estimate, not a billing guarantee. Set provider-side project budgets as well; there is no global billing reconciliation or automatic price lookup. Disabling the provider gives deterministic zero external cost.

No raw provider output, question or key is logged by this code. Common email and Bearer/API-key patterns are redacted before storage and provider transmission; this is not complete PII detection. Users must avoid sensitive personal data. Only generated aggregate statements and a redacted question go to the provider, not raw rows, actor identifiers or complete documentation. Actor IDs are hashed in the daily budget ledger. Ensure upstream access logs do not record bodies/tokens.

18.5 Retention, operations and limits

Sessions expire after seven days; reads reject expired sessions immediately. An hourly maintenance task physically deletes expired sessions/messages. Budget rows older than eight days are removed on reservation. Database backups follow the deployment's separate retention policy. Ownership and scope checks apply to stored answers; source-record edits are not retroactively reflected in history, so an answer is a dated snapshot. Delete a conversation to remove its stored messages.

Answers are buffered until all evidence IDs validate. This implementation does not stream unvalidated provider tokens. Loading state is accessible; there is no fake typing animation. Provider requests have an eight-second timeout, Java has ten seconds and the web adapter twenty seconds. Actual local/CI measurements are recorded separately; these timeouts are not latency SLOs.

18.6 Validation

Python tests cover authorization, schema rejection, injected provider responses, unknown/repeated statement IDs, credential-free fallback and owner-separated budgets. Java tests cover ownership predicates, redaction and allowlisted document fingerprints; Testcontainers exercises stored evidence, session deletion and role revocation against PostGIS. Compose smoke checks Java to Python and Next to Java paths. Playwright exercises error/empty states, suggestions, deletion, mobile width and axe accessibility. Browser contract fixtures intentionally contain no invented analytical facts. An actual external provider evaluation and a formal security audit remain pending.

18.7 Architecture

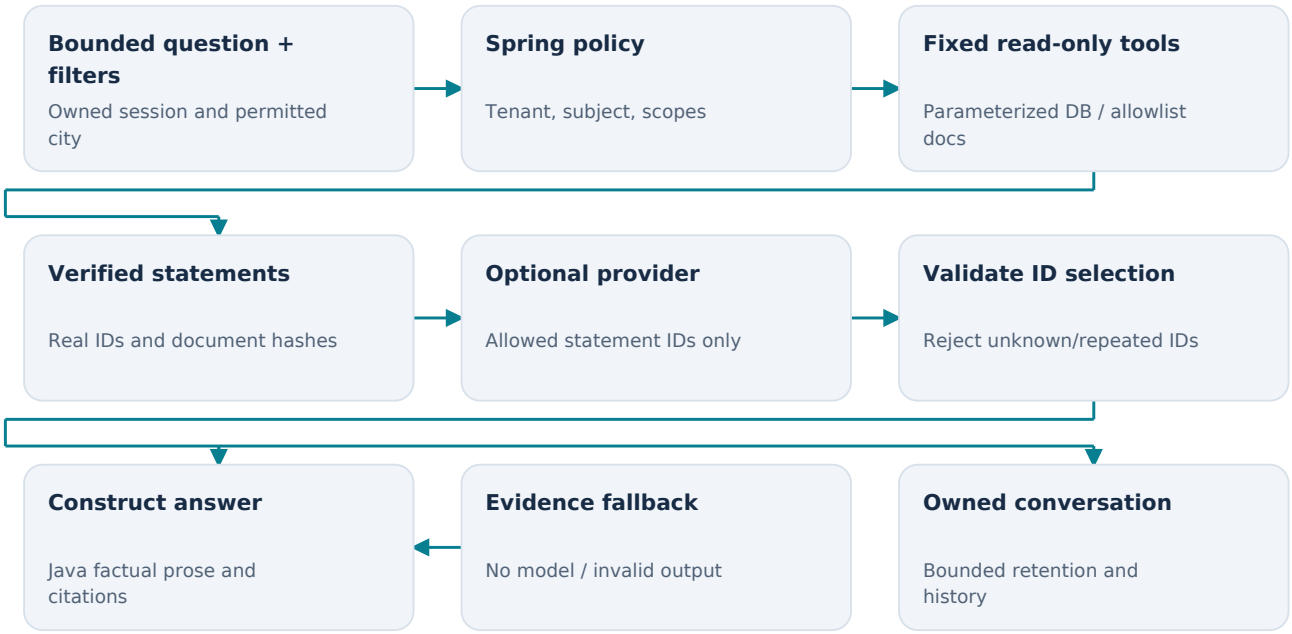
Threat controls include tenant/actor predicates against identifier guessing, server-side membership checks against role spoofing, allowlisted files against path traversal, parameterized SQL against injection, ID-only model output against prompt injection, explicit local-preview configuration against anonymous token delegation, same-origin web mutations, bounded payload/context/session/request/token usage and no raw source HTML rendering. Compromised server credentials, provider compromise, operator price misconfiguration and incomplete source data remain operational risks.

18.8 Screenshots and rendering measurements

Mobile view. These captures show the locally running production build before retrieving any analytical records. No external provider or city intelligence is implied. Reproduce with npm run build followed by node scripts/measure_web.mjs; output is in web-performance.json. Browser contract tests also exercise empty responses and error states with explicitly synthetic test fixtures.

Implementation sources: docs/pulseai.md

Figure 09 - PulseAI: evidence selection, not generated claims



Retrieved text is untrusted and cannot select arbitrary SQL or tools. Both Python and Java validate provider-selected IDs, while factual prose and citations are constructed from permitted evidence. Missing data remains unknown. The model does not infer protected tenant filters from natural-language instructions or autonomously change dashboards.

Generated from editable source/diagrams.json and the inspected implementation.

19 Opportunity discovery and resource workspace

The /opportunities and /optimization routes share seven dashboard views: scored opportunities, coordinate gap overlay, inventory simulator, district/city comparison, scenario comparison, operating-budget investigation and decision explanation. Synthetic retail, logistics and public-service fixtures are in data/sample/decisions. They imply no access to real-time city data or guaranteed profit.

19.1 Run and reproduce

Start the stack with python scripts/setup-env.py (only for a new .env) and docker compose up --build -d --wait. Existing .env files need a distinct random DATABASE_PASSWORD for the new restricted runtime role. Open /opportunities, choose a fixture and **Run & save via API**. Server sessions/scenarios require an authorized identity; explicit loopback local Compose uses its development identity. For synthetic-only host development, enable both PULSEAI_LOCAL_PREVIEW=true and DECISION_LOCAL_PREVIEW=true with the configured AI URL/token. Local evaluation uses SciPy but has no durable server audit trail. Real snapshots require Java.

```
python -m pytest services/ai-engine/tests/test_decisions.py
python scripts/decision_demo.py
```

Use the AI virtual environment containing the locked dependencies. The demo emits measured synthetic evaluations and the canonical scenario JSON schema. Record solver version and hashes; numerical equality across versions is not promised.

19.2 Domain and assumptions

One scenario fixes a unit, currency, time zone and positive half-open interval up to 366 days. It supports up to 30 district/category groups, ten stocks and 300 lanes. Demand, existing availability and forecasts remain separate. Categories are canonical commodities/service quantities, not automatically inferred industry taxonomies. Retail uses kg in its fixture; public service uses integer service counts. No currency conversion or aggregation of unlike units occurs. Multi-city scores weight groups equally and disclose missing groups. Cross-scenario numerical comparison rejects different periods/units/currencies and labels source mode separately.

Operator risk, accessibility, density reference, priorities, costs, capacities, coverage floors, multipliers and seasonal baselines are assumptions. Opportunity score is a weighted mean of available gap share, inverse assumed risk, accessibility, inverse relative business density and bounded seasonal change. Missing features are listed and remaining weights renormalized. Missing demand/availability means no score; missing is never zero. Factor contributions explain every score. These are heuristics, not calibrated investment probabilities, revenue forecasts or business valuations.

SciPy HiGHS solves a transport allocation with variables per permitted source/group edge and one minimum-coverage variable. Stocks constrain outbound quantities; receiving capacity and positive gaps cap inbound quantities; costs constrain total budget. Category matching prevents rice becoming water. Per-district and global coverage floors are hard lower bounds, never silently relaxed. Continuous allocations are divisible; integral=true uses SciPy MILP for whole units. Allocation never dispatches.

Objectives are total throughput, priority-weighted quantity coverage, minimum cost, max-min fairness, or a configurable normalized weighted combination of throughput, cost and fairness. Secondary minimization reduces cost while retaining the primary objective within tolerance. Cost-only objectives can correctly allocate zero without positive coverage floors. The engine independently checks residual constraints and integrality. Each solve is bounded to two seconds; primary/secondary and alternatives can require multiple solves. Feasible objective comparisons are not a complete Pareto frontier. Time-limit, solver-error, insufficient-data and infeasible results have no fabricated allocation. Infeasibility diagnostics are not an IIS proof.

Stocks are explicit inventory, never inferred from observed surplus flows. Registered or assumed direct lanes can constrain per-unit cost, capacity and maximum travel minutes. Without supplied lanes, travel constraints return unsupported_constraint. This does not solve vehicle tours, congestion, schedules, time windows or a real road graph. The map is a coordinate overlay without a street basemap or inferred routes. Population accessibility, profitability, fixed capital costs and legal permits are outside this engine. Expansion suggestions are investigation candidates only.

19.3 API and provenance

All /api/v1/decisions routes require signed analytics/data scopes and active tenant membership; writes additionally require analytics write. Ownership is tenant+subject.

Method	Route	Behavior
POST	/snapshots	Authorized cities/categories/unit/from/to to immutable observations and latest reported inventory
POST	/networks	Register direct lanes against tenant source metadata and an explicit license
POST	/scenarios	{scenario: <schema>} plus Idempotency-Key to validate, solve, save input/result and audit
GET	/scenarios?page=0	Owned scenario metadata, 20 per page
GET	/scenarios/{id}	Owned immutable result with input and associated snapshot/network
GET	/scenarios/{id}/export	Authorized JSON attachment
POST	/scenarios/{id}/reproduce	Re-evaluate frozen input with current engine and report hash matches

The AI internal route is POST /v1/decisions/evaluate with service Bearer credential and tenant UUID. FastAPI generates OpenAPI and validates the scenario JSON schema. Spring rejects missing/invalid fields and owns authorization/provenance. Internal service credentials do not authorize a browser identity. No arbitrary SQL executes.

Authorized snapshots overwrite client-supplied measured quantities on every run; only explicit business assumptions remain editable. Snapshot aggregation requires one data mode and interval duration; missing temporal coverage or observation sides remain unknown. Inventory is

latest per business/category/unit within the window; reported inventory age and source completeness require review. Snapshot source IDs and inventory/observation references are stored. Forecast association requires an explicit operator mapping from stored forecast to registry zone/category/units and daily horizon; no automatic geographic or unit inference is asserted. Sources can be incorrect even when authorized. Fingerprints attest reproducibility, not truth.

Saved scenarios include canonical input, output, engine/SciPy version, input/result SHA-256, timestamp, subject and audit event. Compute time is excluded from result fingerprints. Export contains the frozen snapshot/network when retained. Defaults retain scenarios 90 days and unreferenced snapshots 30 days; deployment settings and backups require separate review. There is no production workload benchmark or investment success claim.

Implementation sources: docs/decision-engine.md

20 Optimization algorithms and measured cases

Each permitted source-to-district lane has a nonnegative allocation x; z represents minimum coverage. Outgoing allocation cannot exceed stock. Incoming quantity is bounded by positive demand gap and receiving capacity. Supplied stock/lane unit costs determine the budget constraint. Category-incompatible and excluded travel lanes are not permitted variables.

The hard coverage floor requires availability plus received quantity to reach the greater of global and district coverage times demand. Infeasibility does not silently relax this policy. Missing demand or availability returns insufficient_data; travel limits without actual lane times return unsupported_constraint. No road network or travel estimate is invented.

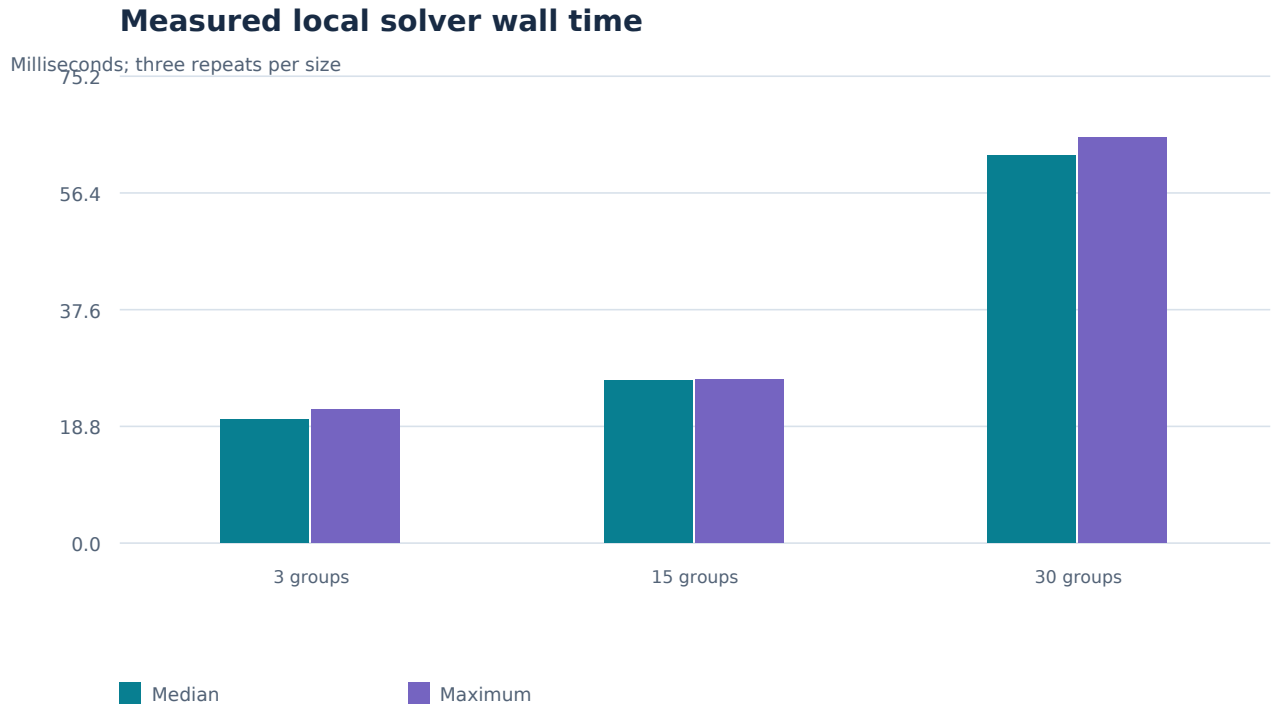
Throughput maximizes allocated units, coverage weights relative district coverage/priority, fairness maximizes z, cost minimizes expense under hard constraints, and balanced normalizes configured objective weights. Cost can validly allocate nothing when no floor requires positive allocation. Secondary cost minimization removes expensive ties within tolerance. Continuous cases use linprog/HiGHS; integral cases use milp with z continuous, bounded time and independent residual/integer checks.

Opportunity scores combine configured gap, forecast growth, seasonality, accessibility, density and risk. Missing optional features renormalize available weights instead of inventing values. These scores are not success probabilities. Multi-city comparison requires matching unit, period, currency and evidence definitions. Canonical inputs/fingerprints aid reconstruction; they do not prove the assumptions true or dispatch a plan.

Implementation sources: services/ai-engine/citypulse_ai/decisions.py; docs/reports/decision-evaluation.json

Synthetic case	Status	Allocated	Cost	Compute ms
retail	optimal	159.99999112796212	799.9999556398107	106.352
logistics	optimal	79.99990823529413	479.9994494117648	90.8
public-service	optimal	160.0	480.0	129.34

Groups	Sources	Candidate lanes	Repeats	Median ms	Max ms
3	1	3	3	19.857	21.489
15	5	75	3	26.242	26.283
30	10	300	3	62.432	65.364



Measured local examples; three warm/cold-mixed runs per benchmark, not production SLOs or investment outcomes.

21 Authentication, authorization and security architecture

This is a development system with implemented safeguards and documented deployment requirements. No penetration test, independent smart-contract audit, SOC 2 report, or compliance certification is asserted.

21.1 Authentication, sessions and account lifecycle

Set WEB_ORIGIN to an exact HTTPS origin, OIDC_ISSUER_URI, a confidential OIDC_CLIENT_ID/OIDC_CLIENT_SECRET, and registered redirect https://your-origin/api/auth/callback. openid-client performs discovery and authorization-code processing; code uses S256 PKCE, nonce, state, single-use five-minute Redis flow handles and explicit signature verification. Redirect URIs are server configuration, never request parameters. Both identity and access tokens must be accepted: the API independently validates the access token and active membership before a session is issued.

MFA is implemented by the identity provider. Set OIDC_REQUIRED_ACR to its exact documented assurance value and configure the provider to place it in both ID and access tokens; both boundaries reject mismatches. Merely requesting acr_values does not enable MFA. No real

provider enrollment was performed by this change. Use phishing-resistant factors where supported. IdP recovery, verified contacts, password policies, enrollment and credential resets remain IdP responsibilities.

Sessions use 256-bit random opaque handles in Secure, HttpOnly, SameSite=Lax, host-only __Host- cookies. Access tokens are encrypted with AES-256-GCM in Redis, never localStorage or browser-readable cookies. Set SESSION_ENCRYPTION_KEY to a base64-encoded random 32-byte secret and SESSION_REDIS_URL to a private TLS Redis endpoint. A deliberately explicit SESSION_REDIS_ALLOW_LOCAL=true exists only for local test stores. Session and token lifetime is capped at one hour, with no refresh token stored. Expiry requires a new login. API membership rechecks reject disabled users immediately even if a JWT/session has not expired. Redis failures fail closed. Login attempts have a shared 100/minute admission limit; enforce per-IP limits at the trusted edge as well. This code does not trust forwarded IPs for identity.

POST /api/auth/logout requires the exact origin, deletes the server session, and clears cookies. It does not claim to terminate the global IdP session. With no stored refresh token there is no refresh-token revocation step. For incident-wide termination invalidate the Redis session namespace and rotate the encryption key. Provider-issued bearer tokens remain valid until expiry unless the account is disabled or the IdP performs supported revocation. Configure short token lifetimes.

Accounts are preprovisioned by a tenant administrator against a real IdP subject; there is no public self-registration or trust in UI demo roles. Only an administrator can change membership roles/status, and writes are audited. Lifecycle updates take a per-tenant transaction lock and cannot remove the final enabled administrator or disable one's own account. The initial administrator must be provisioned through an operator-controlled bootstrap SQL change; no production bootstrap endpoint exists.

21.2 Access matrix

All private data queries derive tenant identity from the validated JWT. A database membership can narrow signed scopes, never expand them. Additional object ownership checks apply to conversations, decision scenarios, snapshots and lane networks.

Actor	Registry	Data	Analytics	Tenant administration
Citizen	Read	None	None	None
Business owner / provider	Read	Read/write	Read	None
Analyst / planner	Read	Read	Read/write	None
Administrator	Read/write	Read/write	Read/write	Read/write

PulseAI requires both data and analytics read scopes; its city picker additionally requires registry read. Decision execution requires analytics read/write and data read; source/network registration also requires data write. Scenario exports remain owner-scoped even for another administrator. Administrative tenant-wide operations must use their explicit existing routes. Tenant administration is not cross-tenant platform administration. No default superuser scope is accepted from the browser.

21.3 Input, output and API controls

Web adapters bound streamed request bytes before buffering: 8,000 for PulseAI, 200,000 for forecasts and 512,000 for decisions. Limits also apply when Content-Length is absent or understated; oversized streams are canceled and return 413. UTF-8 byte and chunk-boundary regressions cover this boundary.

Spring validates DTOs, rejects unknown JSON properties, bounds parser nesting, number and string sizes, restricts multipart files to one MB/500 JSON observations, and enforces a streaming body bound even when Content-Length is absent. Local CSV/GeoJSON ingestion is bounded to ten MB/100,000 rows, rejects duplicate CSV headers, ragged rows and unknown PII-prone columns. It does not execute formulas, archives or documents. Spreadsheet export neutralizes formula prefixes including leading whitespace. Never open unsanitized source CSV files in spreadsheet software.

Server-side parameterized SQL, UUID references, enum/sort allowlists and explicit geographic/temporal bounds constrain data access. Retrieval is read-only for PulseAI; the model cannot execute SQL. Provider output contains allowed evidence IDs, not arbitrary factual prose or citations. Prompt-injection regressions cover malicious sources, invalid IDs and provider output. Missing data remains unknown. No external document URLs or user-supplied filesystem paths are followed by retrieval.

Authenticated Redis counters impose 180 requests/minute per tenant/subject, ten for PulseAI and ten for decisions. Local solver admission and per-instance Java decision semaphores bound expensive computations; solve time and data-size caps apply. These are not volumetric DDoS protection: enforce byte/time/connection and pre-auth rate limits at the ingress. Idempotency receipts protect retried business writes.

Web routes use shared auth resolution; anonymous development-token delegation is disabled unless PULSEAI_LOCAL_PREVIEW=true, and automatically disabled when an OIDC issuer is configured. Local Compose is explicitly local preview on loopback. Never publish it as a production deployment. The synthetic/manual solver adapter additionally requires DECISION_LOCAL_PREVIEW=true and cannot retrieve tenant data. Same-origin checks and SameSite cookies protect browser mutations; bearer-only Spring APIs retain CSRF protection except for /api/v1/ requests explicitly supplying a Bearer Authorization header. Bearer credentials still require independent validation; the exemption grants no access. The API does not authenticate session cookies. CORS denies cross-origin use by default. Configure the UI and API behind the same origin/private boundary; there is no wildcard credentialed CORS exception.

Next generates per-request CSP nonces and renders dynamically. Scripts require a nonce; objects, external base URIs, framing and external form submissions are denied. Inline styles remain permitted for the existing chart/map libraries; this is a documented residual CSP limitation. Permissions Policy disables camera, microphone, geolocation and payment. HSTS is emitted when an HTTPS WEB_ORIGIN is configured; add edge TLS enforcement and HSTS after verifying the domain. API headers use Spring's defaults and restrictive API CSP. Errors omit secrets and stack traces.

21.4 Database, containers and CI

Compose initializes a citypulse_app role without superuser, role creation, database creation or replication. Migrations run with separate owner credentials. Public schema creation is revoked; runtime gets DML only, no schema DDL. Flyway callback revokes runtime access to migration history and mutation of audit events. Runtime can insert/read audit events, while database owners can still alter them. For tamper-evidence ship logs to independently controlled append-only storage; no such external log service is claimed here. Existing tenant predicates are application controls; PostgreSQL row-level security is not claimed as implemented.

Application containers run as non-root with read-only root filesystems, dropped capabilities, no-new-privileges, bounded PIDs and temporary /tmp. AI artifacts have a dedicated writable volume. Redis and PostGIS need writable persistent storage and are not asserted to have immutable root filesystems. Local ports bind loopback. Production must remove public database/service ports, use private networks/TLS, separate migration jobs and external secret delivery.

CI uses immutable action revisions, default read-only repository permissions, no production secrets on pull requests and narrowly scoped security-events write for CodeQL. Dependency/secret checks, Python/Java/web tests, prompt injection and Solidity tests run without wallet funds or provider credentials. Trivy scans source secrets, records configuration findings and gates fixable HIGH/CRITICAL findings in all three runtime images. --ignore-unfixed is explicit: unresolved vendor findings still require review. No runtime image is published by these workflows.

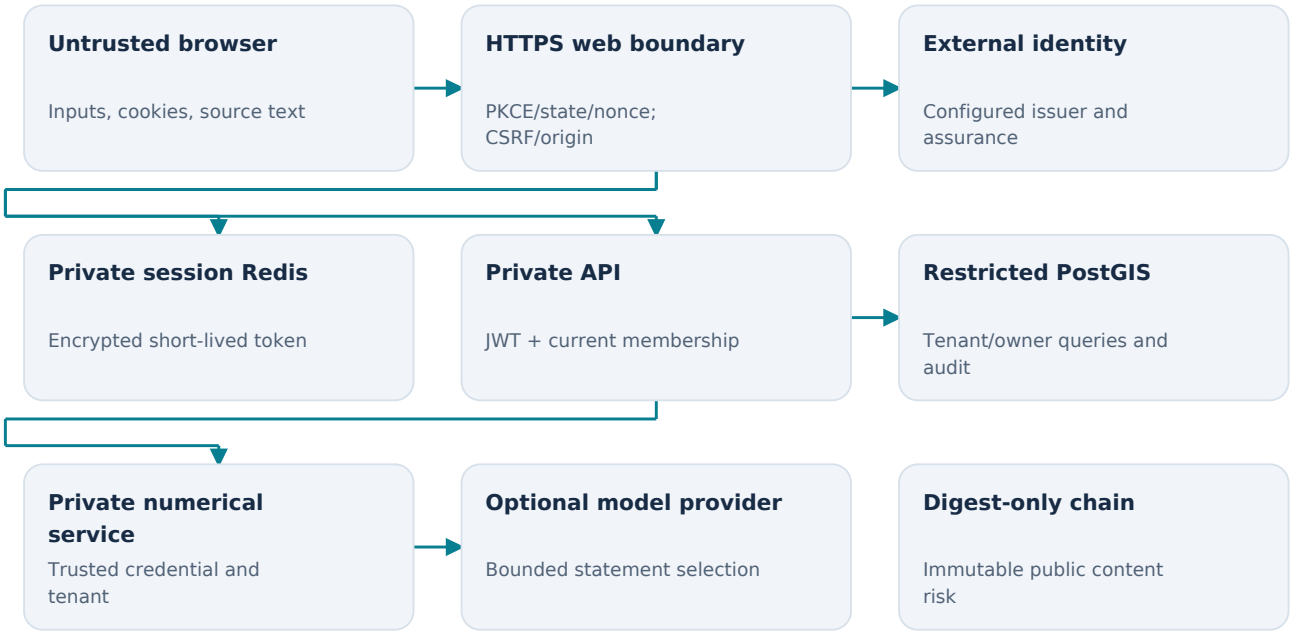
21.5 Retention and residual risks

PulseAI expires at seven days and purges hourly. Decision scenarios default to 90-day deletion and unreferenced snapshots to 30 days via SECURITY_SCENARIO_RETENTION_DAYS/SECURITY_SNAPSHOT_RETENTION_DAYS (1..3650). Backup expiry is separate. No automatic tenant data deletion, legal-hold system, geographic data erasure, IdP management API, audit-retention deletion or SOC controls attestation is claimed. Avoid unconsented personal data and do not put it on-chain. Blockchain immutability prevents deletion; hashes can also reveal correlations.

See threat model, operations, control mapping, scan evidence, and disclosure policy.

Implementation sources: docs/security/architecture.md

Figure 10 - Security data flow and trust boundaries



The layers do not trust browser roles or model instructions as authority. JWT validation, current membership and tenant/owner predicates are independent. Explicit Bearer API requests are exempt from CSRF, while cookie-only/non-API mutations retain protection. TLS, Redis ACLs, IdP setup, edge quotas and real MFA enrollment still require deployment evidence.

Generated from editable source/diagrams.json and the inspected implementation.

22 STRIDE threat modeling

Scope: browser, Next BFF, OIDC, Java, FastAPI, PostGIS, Redis, ingestion, CI and local EVM attestations. The trust-boundary diagram is in architecture. Assume Internet-controlled request bodies, malicious retrieved text, guessed UUIDs, stolen short-lived tokens, compromised dependencies and administrator mistakes.

STRIDE	Concrete threat	Implemented control	Residual / verification
Spoofing	Stolen identity / forged tenant / login CSRF	PKCE, state, nonce, signed tokens, issuer/audience/lifetime/tenant checks, active DB membership, Secure opaque session cookie, optional exact ACR	IdP setup, recovery and MFA enrollment are external; session/claim tests are not proof of real-provider deployment

STRIDE	Concrete threat	Implemented control	Residual / verification
Tampering	Change snapshot values or cross-tenant references	Server-derived immutable snapshots, tenant+owner predicates, fixed SQL, DTO/parser limits, idempotency, input/result hashes	Authorized providers may still report incorrect facts; hashes prove integrity, not truth
Repudiation	Deny role change, allocation or attestation	Transactional sensitive-action audits, generated request IDs, model/solver version and scenario fingerprints, chain receipts	DB owners can alter logs; external immutable log sink remains a deployment requirement
Information disclosure	Browser service-token delegation, leaked sessions, PII in logs	Explicit local mode, shared auth helper, AES-GCM server sessions, no refresh-token persistence, no body/token/IP logging, source minimization, no-store, scope intersection	Bearer theft remains possible; backups, Redis ACLs/TLS and edge logs require operator configuration
Denial of service	Deep JSON, huge import, arbitrary model calls, expensive optimization	Streaming size bound, CSV limits, nesting caps, Redis request limits, bounded solver and inference concurrency, provider budgets/timeouts	Pre-auth ingress DDoS controls and infrastructure quotas must be deployed; known developer dependency advisory is documented
Elevation of privilege	UI persona spoof, stale role, arbitrary SQL/tool, last-admin removal	Server method scopes, per-request membership narrowing, tenant/owner queries, ID-only LLM responses, lifecycle lock	Tenant administrator can still grant roles; separation of duties and approval workflow are not implemented

Specific abuse cases to test: unauthorized reads/exports, foreign city/source IDs, disabled memberships, malformed/missing tenant claims, MFA mismatch, expired session, replayed login flow, invalid callback state/nonce, cross-site mutation, malicious CSV headers, injection instructions in sources, malicious contract callers and duplicate attestations. Existing Solidity tests cover roles and replay/duplicate paths; a professional contract audit and full penetration test remain pending.

Implementation sources: docs/security/threat-model.md

23 Illustrative control mapping, not certification

This table is an educational engineering exercise. It is not a SOC 2 report, attestation, legal opinion, certification or statement that controls operated over an assessment period. Independent organizational evidence would be required.

Illustrative area	Engineering mechanism	Evidence / remaining work
Access control (CC6-style)	OIDC, optional ACR, scoped memberships, tenant/owner predicates, final-admin guard	Automated regression tests; real IdP/MFA rollout and access-review records pending
Change management (CC8-style)	PR workflow, protected main, immutable CI actions, dependency updates	GitHub check results; separation of duties/approval records pending
Monitoring (CC7-style)	Request IDs, minimal security logs, sensitive-action audit events, scanners	Automated scan reports; operational alert routing and immutable log sink pending
Risk assessment (CC3-style)	STRIDE threats, residual-risk register and trust boundaries	Threat document; organization-level risk acceptance pending
Availability (A1-style)	Bounded timeouts/requests, health checks, constrained workers	Local/CI tests; capacity planning, incident metrics and restore drills pending
Confidentiality (C1-style)	Token encryption, TLS deployment requirements, PII minimization, runtime DB roles	Session tests and SQL checks; managed key/backup deployment evidence pending
Data lifecycle	Expiring conversations and configurable scenario/snapshot retention	Code and integration tests; legal holds, tenant erasure and backup expiry pending
Incident handling	Containment, recovery, rotation and disclosure runbooks	Written procedure; tabletop and production exercise evidence pending

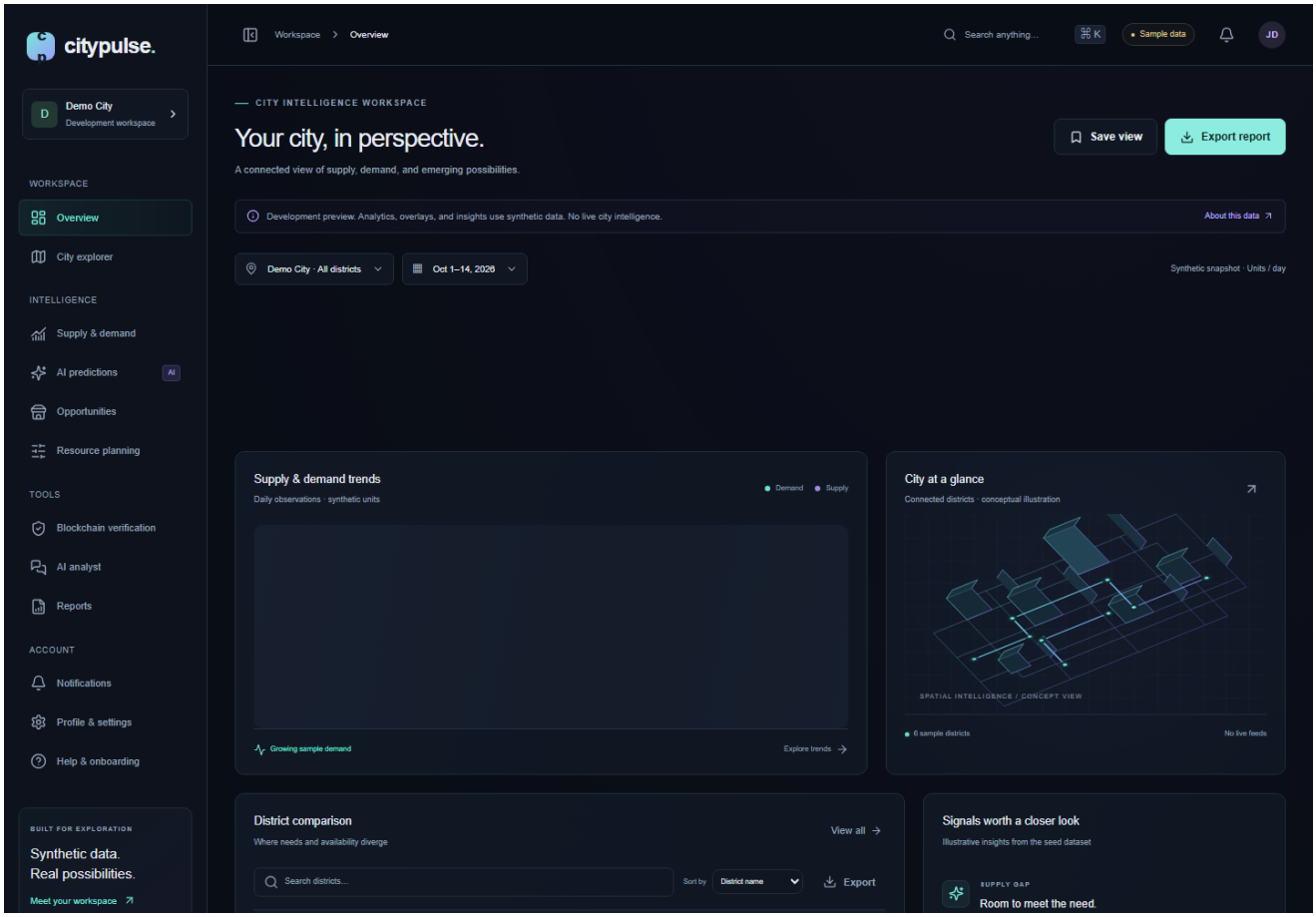
Implementation sources: docs/security/controls.md

24 UI/UX and real application screenshots

The figures are actual Chromium captures, not generated mockups. Historical README captures use labeled synthetic data, and the decision view follows a real local solver call. The verification capture preserves its available/unavailable evidence state. Settings captures show the corrected theme behavior from issue #22 / PR #23. Local Java/PostGIS startup was unavailable; Linux CI supplies full-stack evidence. Screenshots do not prove real city coverage, production hosting or a public-chain deployment.

Implementation sources: docs/assets/readme-provenance.json; apps/web/e2e/theme.spec.ts

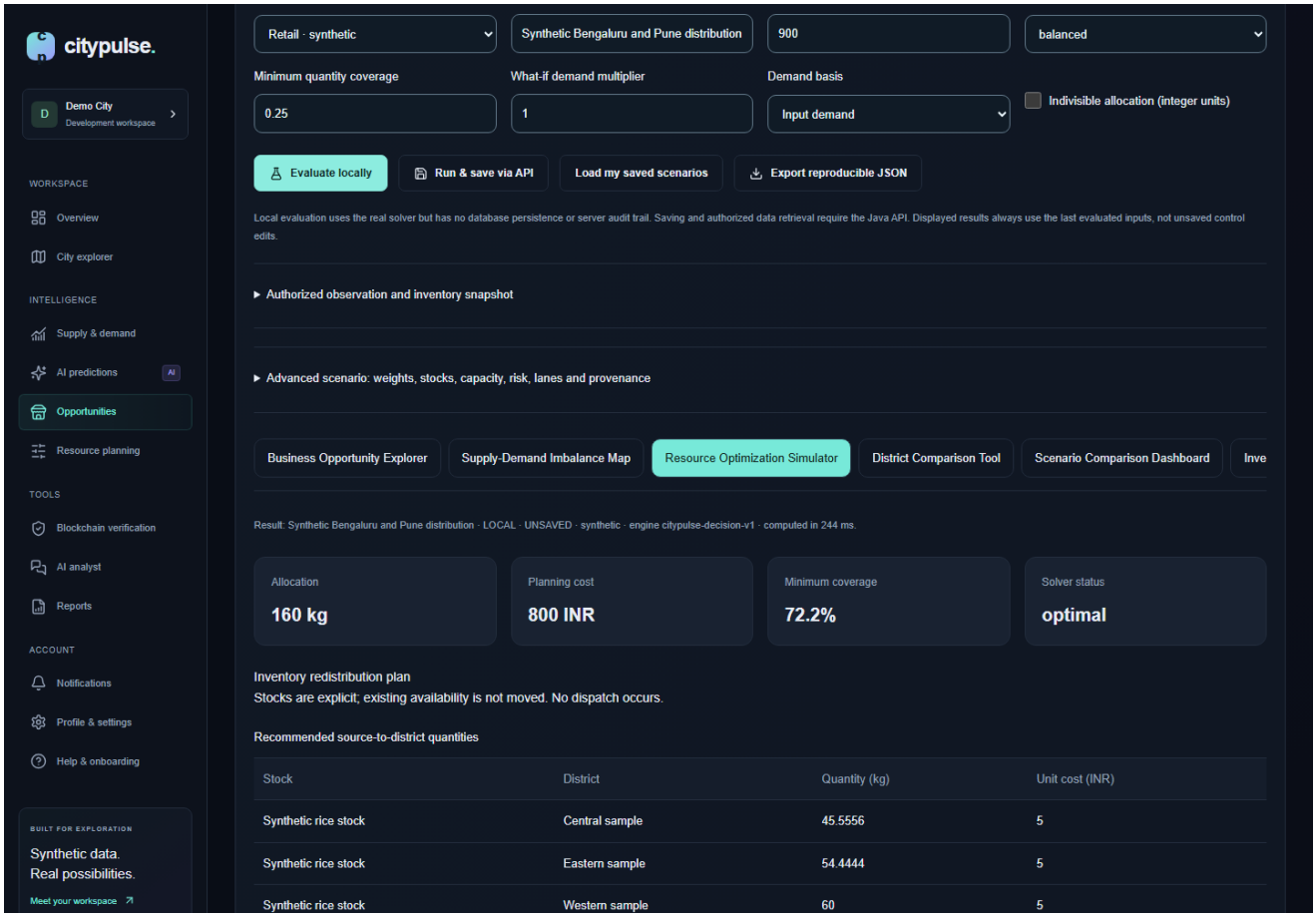
Application evidence



Actual local synthetic analytics dashboard; not live city intelligence.

Source: docs/assets/readme-dashboard.png | Actual Chromium capture. Synthetic data and unavailable states are preserved.

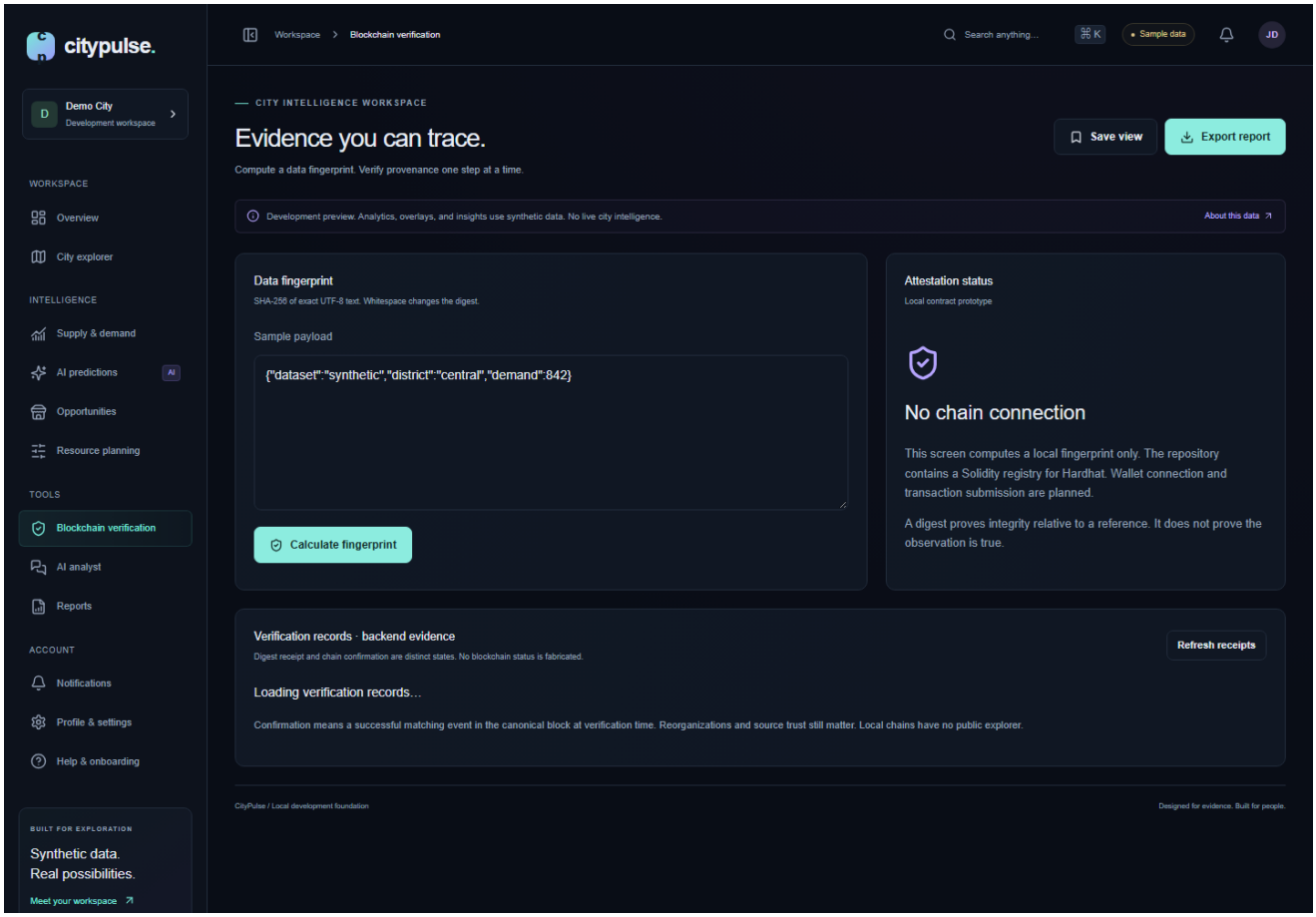
Application evidence



Actual local synthetic SciPy allocation; a local run is unsaved and has no Java audit persistence.

Source: docs/assets/readme-decision.png | Actual Chromium capture. Synthetic data and unavailable states are preserved.

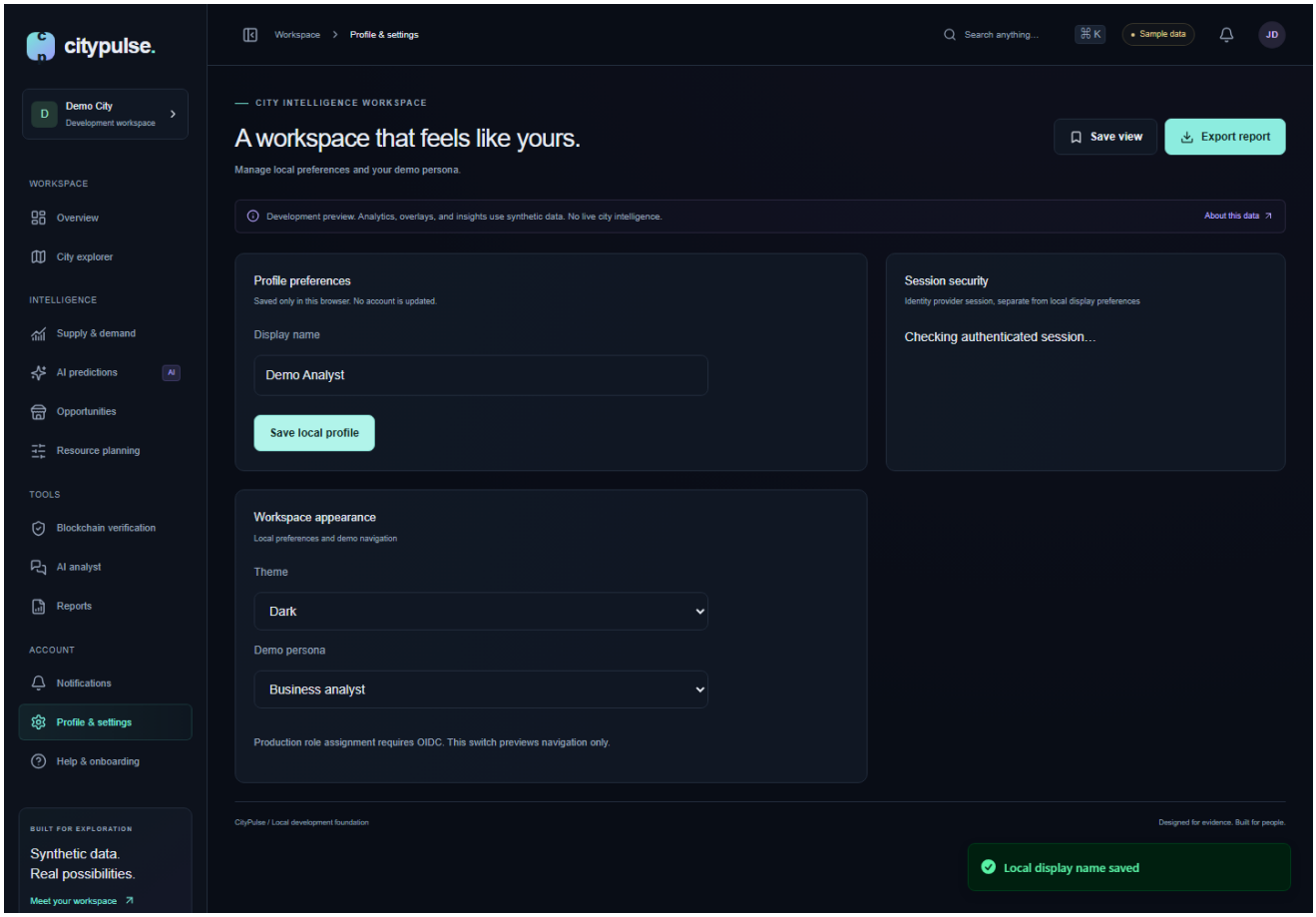
Application evidence



Actual verification center evidence state; no public-chain deployment is implied.

Source: docs/assets/readme-verification.png | Actual Chromium capture. Synthetic data and unavailable states are preserved.

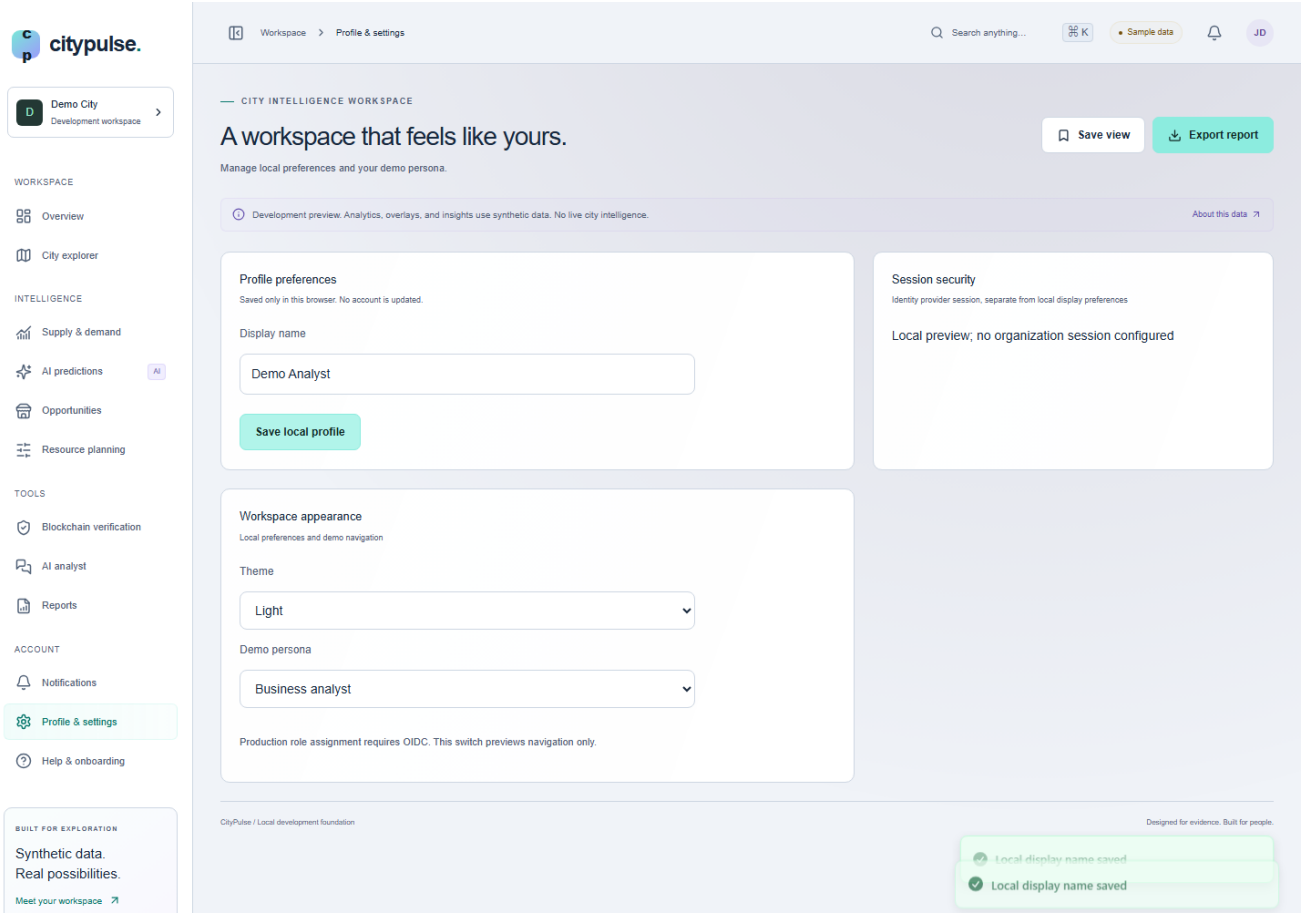
Application evidence



Corrected dark settings and success notification from the actual running application.

Source: docs/reports/diagrams/settings-dark.png | Actual Chromium capture. Synthetic data and unavailable states are preserved.

Application evidence



Corrected light settings; portal and notification theme is shared at document level.

Source: docs/reports/diagrams/settings-light.png | Actual Chromium capture. Synthetic data and unavailable states are preserved.

25 Infrastructure, cloud and progressive delivery

25.1 Implemented versus designed

Stage	Implemented code	Verification boundary
A	Web/API/AI, ingestion CLI and Hardhat Dockerfiles; PostGIS/Redis, seed migration, probes and optional chain	Linux Compose CI; local Docker engine unavailable
B	Pinned CI actions, application/security/platform gates, CycloneDX SBOMs, manual SHA-tag packaging	PR workflows; GHCR publishing is manual and has not been dispatched
C	AWS network, data and ECS Terraform modules, dedicated migration task	Provider validation and mocked plans only; no AWS apply
D	AI/API metrics, W3C traces, redacting collector, Tempo, Prometheus alerts and Grafana dashboard	Optional Compose telemetry smoke; cloud collector and pager delivery require rollout
E	Stateless web/API scaling limits, connection budget, bounded k6 workload and batch client design	Small synthetic workloads; no production capacity certification

25.2 Local startup

Install Docker with Compose v2, Python 3.12+ and Git. Run python scripts/dev_stack.py up --chain --observability from the root. Existing .env is preserved; add a distinct GRAFANA_ADMIN_PASSWORD if upgrading an older environment. The launcher checks Docker, builds/waits and runs cross-service smoke checks. It does not install Docker or change host security settings. Use the same optional flags with down; volumes remain. Ingestion is a finite CLI, not an invented daemon; reporting/admin are existing API/UI boundaries, not separate runnable services.

python scripts/data_pipeline.py demo produces deterministic data, quality/dead-letter/checkpoint outputs. python scripts/data_pipeline.py publish idempotently publishes through the authorized API. To record actual job success/failure in local monitoring, set PUSHGATEWAY_URL=http://127.0.0.1:9092 and run python scripts/ingestion_job.py demo; the wrapper preserves the original process exit status even if metrics fail.

With the chain profile running, execute docker compose run --rm --no-deps -e LOCAL_CHAIN_RPC_URL=http://chain:8545 chain npx hardhat run blockchain/scripts/local-demo.ts --config blockchain/hardhat.config.ts --network localhost. The chain resets when recreated; fixed Hardhat accounts are public test fixtures and must never hold real funds.

25.3 CI, artifacts and preview design

Every PR builds/lints/tests the services and scans production dependencies, secrets, source, configuration and runtime images. The security job uploads image/config reports and runtime CycloneDX SBOMs. Artifacts are tied to the workflow run/revision, not mutable coverage badges. The platform job validates Terraform without credentials, runs mock plans and validates monitoring config and the ingestion image. Compose smoke starts the actual optional chain/telemetry stack and checks requests, persistence, privileges, scrape targets and trace ingestion.

The manual Package release workflow runs only on main and uses the release GitHub environment. It rebuilds, gates fixable HIGH/CRITICAL image findings, produces SBOMs, then pushes ghcr.io/<owner>/citypulse-<service>:<commit-sha> using a job-only packages-write token. It records resolved digests; deployment consumes digests, not mutable tags. Configure environment reviewers and branch restrictions before using this workflow. It does not deploy AWS or provision infrastructure. Cloud publishing has not been executed by this task. Local-only chain tooling is not promoted as a production image; its known development advisory remains disclosed.

Preview design: one disposable Compose project per PR/revision, synthetic fixtures only, no production IdP, cloud credentials, databases or signers. Bind previews to a private runner/network, authenticate ingress if sharing, namespace volumes, cap lifetime/resources and remove only that project's resources after review. Fork PRs run unprivileged checks and receive no release credentials. No hosted preview URL or ephemeral cloud deployment is claimed.

25.4 Configuration and promotion

Development uses the explicit local Spring profile and preview flags on loopback. Staging/production require HTTPS, real issuer/audience/client, scoped memberships, private TLS session Redis, encryption key and private service credentials. Keep the local profile/flags off. Secrets Manager injects individual values into ECS; only the migration task receives schema-owner credentials. Configure separate state, secrets, certificates, image digest manifests and databases per environment; never share a tenant fixture DB with production.

Promote a digest manifest after staging smoke, authorization checks, restore evidence and SLO review. AWS provisioning uses separately approved short-lived operator credentials; there is no automatic Terraform apply. See the AWS module guide.

25.5 Rollback

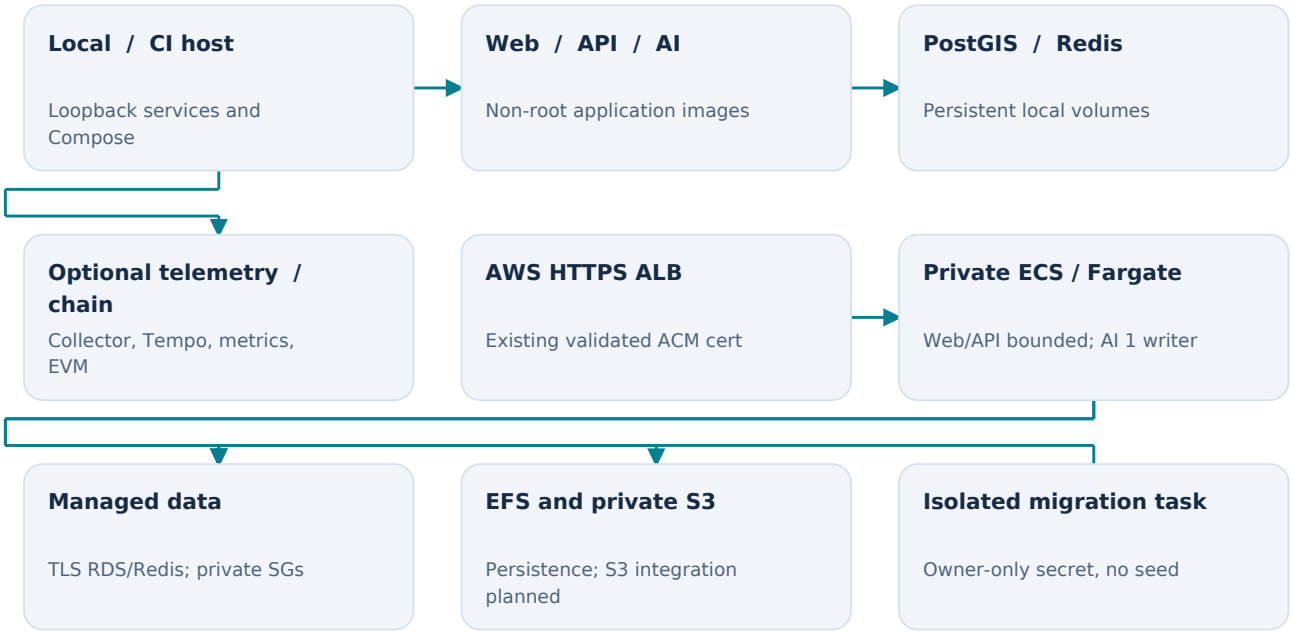
For stateless web/API, record the previous healthy ECS task-definition revision and digest manifest, then roll back via aws ecs update-service --cluster <cluster> --service <service> --task-definition <previous-arn>. Wait for service stability and verify readiness, authorization and a real read/write/replay smoke. Circuit-breaker rollback assists failed task launches; it does not validate business correctness or reverse a migration.

Use expand/migrate/contract schema changes. Keep the previous app compatible throughout a rollout. Never automatically run destructive down migrations. Restore a database only into an isolated replacement and explicitly reconcile writes since the recovery point. AI file/SQLite rollback is stop-the-writer, snapshot/restore the entire state, deploy the previous model/image and verify fingerprints; do not overlap old/new writers. Session-key changes invalidate sessions and may require users to sign in again.

Optional Kubernetes stays planned because running two orchestration stacks adds work without current capacity evidence. Recovery, observability and scaling define operational gates.

Implementation sources: docs/operations/deployment.md

Figure 11 - Deployment: local verification and AWS code



Linux CI validates Compose, while local Docker Desktop was unavailable. AWS modules are provider-schema and mocked-plan validated, not applied. Counts default to zero pending roles, migration and identity bootstrap; data/network resources would still incur costs after an apply. Internal HTTP, NAT egress and writable ephemeral cloud roots are documented follow-up gaps.

Generated from editable source/diagrams.json and the inspected implementation.

26 Observability, monitoring and proposed SLOs

26.1 Signals and trust boundaries

Spring Boot's OpenTelemetry/Micrometer starter traces API requests; W3C trace context is explicitly propagated by outbound Java AI clients. The AI middleware extracts only W3C trace context, records a server span, counts route/method/status and measures durations. Next's optional @vercel/otel instrumentation is activated only with an OTLP endpoint. Export is disabled by default. A collector outage must not fabricate application success; exporter queues/timeouts are bounded.

The local overlay sends traces to a collector with memory limiting/batching and a strict attribute allowlist. It strips URL/query/SQL/body/identity attributes, exception messages/events and span status text before Tempo storage. Resource labels retain service identity, not tenant/user

identifiers. Verify redaction against actual provider traces before production use. Trace sampling is 100% in the local diagnostic overlay, normally 10% in the API; tune cost and privacy together.

API management listens on private port 9091, exposing Prometheus and health; metrics are allowed only on that management port, not through public ingress. /readyz on 8080 checks DB/Redis, /livez checks process liveness. AI /health is process liveness and /ready requires configured service identity; it is not a claim that every model artifact is valid. AI /metrics is internal-only. Never publicly route management or AI endpoints.

Prometheus scrapes API/AI and optional ingestion Pushgateway; Grafana provisions actual throughput/latency/connection/heap/job panels. Tempo stores local trace blocks for 24 hours; Prometheus retains seven days. Grafana requires the generated password. Local docker compose logs aggregates stdout; the AWS ECS task definitions send logs to a centralized CloudWatch group with 30-day retention. API emits ECS JSON; AI emits a minimal JSON request event. Web runtime/startup logs can still be unstructured; no universal JSON-log claim is made. Uvicorn access logs are disabled in the container to avoid raw URL/query logging.

26.2 Alerts and response

Prometheus includes target-down, API/AI 5xx, AI latency and instrumented ingestion-failure rules. They are evaluated locally; no Alertmanager/pager destination has been configured or test-paged. Cloud code adds an ALB target-5xx alarm; provide real alarm action ARNs and verify delivery. Scrape failure is a signal to investigate, not proof of user-visible outage. Missing traffic is not counted as successful requests. Ingestion metrics exist only when the real wrapper runs; missing data is not a successful ingestion.

Triage using generated request IDs/trace IDs, then determine process/network/auth/data impact. Do not paste prompts, source rows, credentials or callback query strings into incidents. See the incident runbook.

26.3 Proposed service objectives

These are initial engineering targets, not achieved SLAs or measured production history.

SLI	Proposed objective	Measurement
Eligible interactive availability	99.5% over rolling 30 days	Successful eligible requests / attempted requests at ingress; include timeouts/5xx, exclude policy-rejected invalid/unauthorized requests
Registry read latency	p95 under 500 ms at a validated load	Ingress request histogram, separate from compute requests
Small forecast/decision latency	p95 under 2 s at documented size limits	Route-specific completed duration plus timeout count
Configured source freshness	Within source-specific schedule plus agreed grace	Last successful published checkpoint versus expected cadence; no schedule means no freshness promise

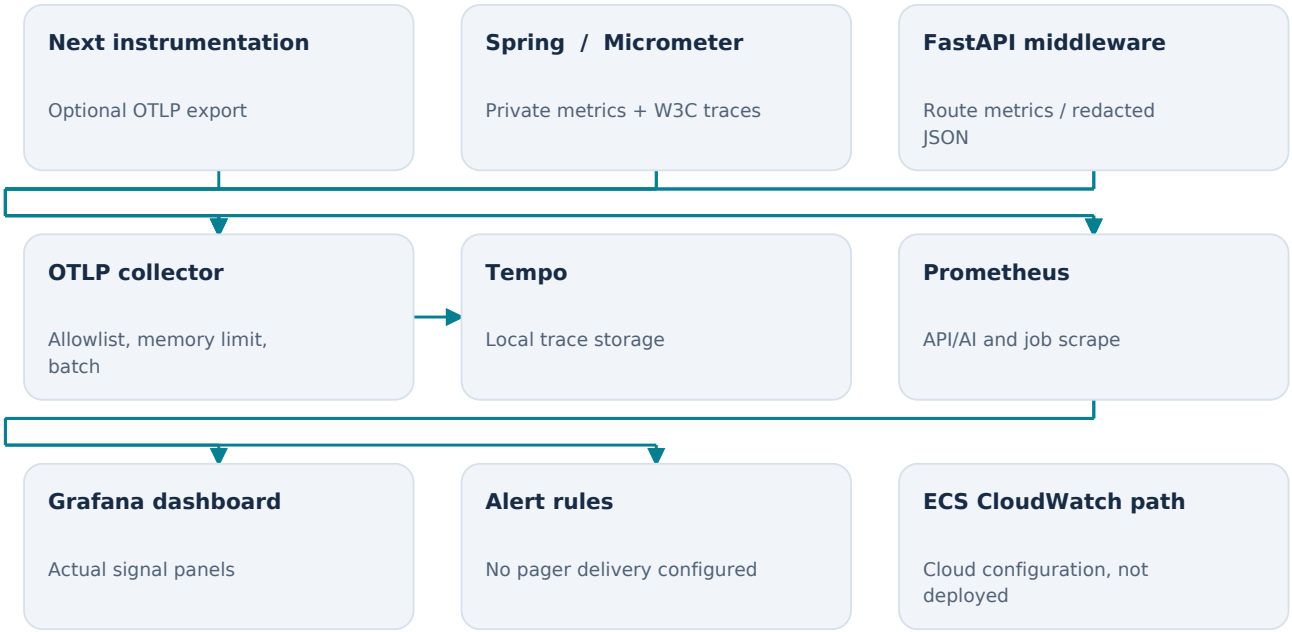
At 99.5%, a 30-day time-based availability budget is 216 minutes; request-based budgets depend on measured eligible volume. Before adopting a budget, define eligibility, minimum volume and

external probe locations. Current dashboards/alerts are foundations, not a complete external SLI implementation.

Policy: investigate fast burn immediately; pause feature releases if the rolling budget is exhausted, except approved remediation. A proposed page requires both 1-hour and 5-minute burn above 14.4x; a proposed ticket uses 6-hour and 30-minute burn above 6x. Implement and test these multi-window rules after real eligibility/traffic is established; the checked-in initial error alerts are simpler thresholds. Record incident impact, release decisions and recovery evidence. No SLO attainment is claimed by a 20-second load smoke.

Implementation sources: docs/operations/observability.md

Figure 12 - Observability and operational evidence



Compose smoke confirms real scrape targets and actual stored spans; this is stronger than container process health alone. Request logs exclude raw query/body/token fields. Local alert rules are not a tested pager route, and proposed SLOs are not achieved service levels. CloudWatch is an ECS log configuration path, not an asserted deployed centralized logging service.

Generated from editable source/diagrams.json and the inspected implementation.

27 Performance, scaling and cost assumptions

27.1 Current limits and next steps

Web/API are stateless with shared Redis sessions, cache/limits and PostgreSQL persistence. ECS starts two replicas of each only after explicit activation, and caps CPU autoscaling at four. A ten-connection Hikari pool means up to 40 API connections at that cap, plus migrators/operators and safety margin. Choose RDS memory/max_connections from measured workload; do not raise pools to hide slow queries. Observe pool wait, DB CPU/IO/locks and query plans first. Existing tenant/geospatial/temporal indexes remain; run EXPLAIN (ANALYZE, BUFFERS) on representative authorized queries in staging without sensitive result logging.

Cache TTL is bounded and tenant keys are preserved. Invalidation must follow writes affecting a cached result; never share tenant-derived cache keys. Measure hit/miss and invalidation behavior before extending TTL. Use keyset pagination/summary tables for large histories only after query evidence warrants the change.

AI uses SQLite/files and a model registry. One task/writer and stop-before-start deployment are deliberate limits. EFS supplies persistence, not a distributed transaction/locking architecture. Before horizontal AI scaling, move job metadata/leases to PostgreSQL, store immutable artifacts in object storage, add idempotent bounded workers and prove concurrent retry/rollback behavior. The optional SQS/DLQ infrastructure has no connected consumer today. No Kafka/Redpanda is introduced without an actual ordering/throughput requirement.

The existing forecast API handles individual versioned requests. Batch callers should submit bounded chunks, limit concurrency to two, reuse stable idempotency keys and record each job ID/error; they must not bypass tenant access or retry permanent 4xx failures. Scheduled retraining remains an operator workflow. A durable batch scheduler, transactional outbox and SQS worker are planned, not implemented distributed processing.

Decision solver limits: 30 district/category groups, ten stock sources, 300 lanes, finite quantities and bounded compute time. Requests report infeasibility/time limits instead of manufactured allocations. Do not infer network routing without actual licensed lane/network input.

27.2 Reproducible k6 workload

Install k6 2.3.0 from its official release and verify its published SHA256. Supply LOCAL_API_TOKEN via an ignored environment source; the workload never prints it.

```
# Existing synthetic retail fixture, actual AI endpoint, no response mocks.
BASE_URL=http://127.0.0.1:8300 TARGET_MODE=ai k6 run tests/load/smoke.js
# API requires a running authorized Java/PostGIS stack.
BASE_URL=http://127.0.0.1:8080 TARGET_MODE=api k6 run tests/load/smoke.js
# Health-only web smoke is a separate, weaker workload.
BASE_URL=http://127.0.0.1:3300 TARGET_MODE=web k6 run tests/load/smoke.js
```

PowerShell uses $env:BASE_URL='...' and $env:TARGET_MODE='...' before k6 run. Set SUMMARY_PATH to retain each JSON summary. Default is one arrival/second for 20 seconds, two preallocated/four maximum VUs. Gates: p95 under two seconds, under 1% HTTP failures, all semantic checks pass and no dropped iterations. These intentionally small smoke gates are not capacity tests or proof of the proposed SLO. Record machine/runtime, duration, route,

concurrency, data mode, warm/cold state, failures and raw summary. Local full-stack load/container tests are blocked by Docker Desktop; Linux CI exercises integration. Measurements from direct AI exclude Next/Spring/network/database latency.

Measured outputs, when produced, are retained under operation reports. Do not substitute an expected performance number when a workload cannot run. Longer soak, spike, tenant-fairness, DB saturation, failover and restore/load tests remain required before production capacity promises.

27.3 Planning cost envelope

No paid resources were created or priced as a binding quote. Illustrative monthly inputs for a 730-hour two-AZ staging design: 4 aggregate Fargate vCPUs/10 GiB at baseline, two NAT gateways, one ALB, Multi-AZ RDS db.t4g.small, two cache.t4g.small Redis nodes, 30 GiB RDS plus backups, EFS/S3/log/trace traffic. At web/API scale cap, CPU/memory and connection spend rise. AI remains one task.

For early budgeting only, reserve roughly **USD 450-900/month**, excluding tax, support, IdP, external LLM calls, heavy egress, large backup growth and cross-region DR. This is a broad engineering allowance, not a measured bill or current regional quote. Before any apply, replace it with a dated regional AWS Calculator estimate and approved usage caps; include per-hour compute, NAT processing/public IPv4, ALB LCUs, storage/IO, logs, data transfer and backup retention. Zero ECS service count does not stop data/network charges. Compare single-AZ sandbox compromises explicitly rather than silently weakening availability settings.

Authoritative inputs: Fargate pricing, VPC/NAT/IPv4 pricing, RDS PostgreSQL pricing, AWS Calculator. Regional rates and service availability can change.

Scaling triggers: sustained web/API CPU over 60% with healthy DB to bounded scale-out; pool waits/DB IO saturation to query/index/cache investigation before replicas; AI queueing/compute latency to size-bound requests and durable-worker design; stale ingestion to source/retry investigation; recurring budget burn to freeze feature rollout and remediate. Validate every trigger against actual telemetry and documented costs.

27.4 Recorded local result

On 2026-10-29, k6 2.3.0 sent 21 real synthetic decision requests at one arrival/second over 20 seconds to the updated FastAPI/SciPy service on loopback. All semantic checks passed; HTTP failure rate was 0%; p95 request duration was 74.68 ms. See the raw summary for averages, maxima, thresholds and the environment. This excludes web/API/DB latency and provides no production capacity estimate.

Implementation sources: docs/operations/scaling.md

Measured k6 field	Actual result
Requests	21
HTTP failure rate	0
Average duration ms	51.793
Median duration ms	46.738
p95 duration ms	74.680

Measured k6 field	Actual result
Maximum duration ms	80.212
Mode / rate / duration	ai / 1 per second / 20s

Bounded smoke workload, not a production capacity or SLO claim Direct AI on a shared developer host; no Next, Java, DB or external-network latency is included.

Historical browser observations

single local Chromium run against production build; synthetic data, browser cache disabled; 1.5-second observation window after heading; map tiles may be unavailable; no production performance claim

Route	DOMContentLoaded ms	FCP ms	Transferred JS bytes
/	8545	8188	251501
/dashboard	180	200	247894
/explorer	233	276	753086
/analyst	2793	3108	251501

28 Reliability and disaster recovery

No production backup or disaster-recovery drill has been executed. This is an operator design plus infrastructure retention settings, not an RTO/RPO guarantee.

28.1 Data and initial planning assumptions

RDS has 14-day automated backup retention, encryption, Multi-AZ and deletion protection; enable cross-region/account backup copies and test PITR before a real availability commitment. Multi-AZ is availability, not protection from logical deletion. S3 has versioning, TLS-only access and noncurrent-version retention; object lock/cross-account copies require a separate legal/operational choice. EFS has an AWS Backup policy but its regional default backup plan, retention and restore access must be inspected and documented after deployment. Redis has seven-day snapshots, but sessions/caches should be recreated rather than restored after compromise.

Proposed starting RPO is one hour for authored data and RTO four hours for a sandbox recovery; neither is proven. AI SQLite/file state requires coordinated stop-the-writer snapshots and consistent model/artifact fingerprints. SQLite online file copying without its WAL/checkpoint is not an acceptable backup.

28.2 Safe rehearsal

• Inventory schema/version, immutable image digests, model versions, secret references and last durable ingestion checkpoints. Record counts/checksums without exporting PII into tickets.

• Take encrypted logical backups through a dedicated read identity; use pg_dump --format=custom --no-owner --no-acl. Put credentials in a protected secret mount/.pgpass, not shell history. Store backup integrity hashes separately; never commit dumps.

• Restore into a new private database/account using compatible PostgreSQL/PostGIS, pg_restore --no-owner --no-acl --exit-on-error, and approved operator credentials. Verify extension/schema versions, constraints, counts and checksums. Reapply least-privilege roles/audit restrictions; runtime must not gain owner privileges.

• Replay only idempotent ingestion receipts/checkpoints after identifying recovery-point losses. Verify tenant and owner isolation, auth, import, forecast/scenario reproduction and export hashes. Verify selected on-chain attestations against restored off-chain bytes; chain history cannot be rolled back by a database restore.

• For AI, stop the single writer, restore a consistent state/artifact pair, reopen SQLite, validate fingerprints and run actual sample inference. Start one task only. Invalidate sessions and rotate affected secrets after an incident; do not recover revoked session handles from general backups.

• Measure actual recovery duration/data loss, verify private networking and TLS, then separately approve DNS/traffic cutover. Retain the former system read-only for reconciliation if safe. Document failures and update the next drill, rather than declaring the plan successful because a snapshot exists.

The security runbook covers containment, disclosure and key rotation; the deployment guide covers compatible application rollback. Routine dev_stack.py down never removes volumes. Destructive volume deletion/destroy is intentionally not part of one-command startup/stop.

Implementation sources: docs/operations/recovery.md

29 Account/key operations and incident response

29.1 Secure deployment checklist

• Use HTTPS for browser, IdP, session store and production service boundaries. Set

exact issuer/audience and HTTPS WEB_ORIGIN; register only the exact callback URI.

• Disable both preview flags. Do not deploy the local Spring profile. Configure

production IdP claims tenant_id, sub, aud, scope, short expiry and optional exact acr. Register two authorized administrators before normal use.

• Generate a random 32-byte session key: openssl rand -base64 32. Deliver it and

client/service/database secrets through a secret manager or mounted runtime secret mechanism. Never bake them into images, commit them, or print them.

• Use a private TLS Redis store, restrictive ACLs and encryption-at-rest. Verify

cookie Secure/HttpOnly/SameSite flags, login/callback failure, membership denial, logout, expiry and optional MFA on the actual provider before enabling traffic.

• Run migrations in an isolated job, then set FLYWAY_ENABLED=false in the runtime

and remove migration credentials. Supply DATABASE_USER=citypulse_app and its distinct password. Confirm runtime cannot CREATE, DROP, UPDATE/DELETE audits or read Flyway history. Do not expose DB, Redis or FastAPI to the Internet.

• Apply edge request-size, connection, body-read timeout and pre-auth/IP limits;

trust no arbitrary forwarded headers. Review CSP and failed auth logs; never log token-bearing query strings, callback codes, passwords, raw prompts or bodies.

• Review all CI artifacts and open dependency advisories. Rebuild and rescan images

after vendor fixes. Confirm no production credentials are available to fork PRs.

• Apply backups, restore drills, retention and legal-hold procedures appropriate

to your data and jurisdiction. No compliance determination is supplied here.

29.2 Account/key management

IdP administrators own onboarding, MFA, recovery and account suspension. The app stores subject/display name and tenant membership only. First verify the real IdP subject, then provision the tenant membership via the authorized admin API. Disabling it immediately removes API access because membership is rechecked. Offboarding must also disable the IdP account and invalidate app sessions. Test this with multiple tenants; do not assume token expiry alone is immediate revocation.

Rotate OIDC client secrets using the provider's overlap procedure, update the secret store, restart web instances and confirm a real login. Rotate the session key by deploying the new value and clearing session:* and oidc:flow:* in the dedicated session Redis database; users must reauthenticate. Only execute deletes against the dedicated session database after verifying its URL/database index. Do not use FLUSHALL on shared Redis. The code intentionally has no old-key decrypt fallback.

For service-token rotation coordinate API and FastAPI atomically; otherwise calls fail closed. For database passwords rotate the runtime role independently of the migration owner, drain/restart pools and verify privileges. Public-chain signers must use a managed signer/HSM or protected wallet. Hardhat keys are local fixtures, never deployment assets. Rotate contract roles through an authorized multisig where supported and verify receipts; on-chain events cannot be erased.

29.3 Backup/restore design

Use encrypted, access-controlled logical backups with pg_dump --format=custom --no-owner --no-acl; use a dedicated backup identity with necessary read rights, not the web process. Keep credentials in .pgpass/secret mounts with restrictive permissions. Require transport encryption, backup integrity hashes, immutable/offline copies and a separate retention schedule. Never commit database dumps. Suggested initial planning targets: daily backups and a monthly isolated restore rehearsal; RPO/RTO must be measured, not promised.

Restore into a fresh private database with matching PostgreSQL/PostGIS versions. Use pg_restore --no-owner --no-acl --exit-on-error, run schema/integrity checks, reapply runtime grants and audit immutability, then compare counts/checksums and exercise tenant-isolation/authentication smoke tests. Do not restore sessions or production signer secrets from a general database dump. Rotate app/session secrets after a breach. Production backup automation and a completed restore drill are not claimed by this repository; record drill duration and recovery gaps separately.

29.4 Incident response runbook

• **Triage:** record UTC incident time, generated request IDs and affected service,

tenant and release. Avoid copying sensitive payloads into tickets or chat.

• **Contain:** remove public ingress or disable the affected route/tenant; suspend

implicated memberships/IdP subjects; rotate leaked credentials and invalidate sessions. Preserve logs before changing retention. Never destroy evidence or assume blockchain deletion is possible.

• **Assess:** identify access boundaries, data types, timeline, secrets and backup

exposure. Use least-privilege forensic access. Determine notification obligations with the responsible organization; no legal deadlines are invented here.

• **Eradicate:** remediate root cause, pin reviewed updates, rescan and add a regression

test. Rebuild signed/reviewed images through trusted CI; do not hot-patch secrets into source or bypass failed required checks.

• **Recover:** restore from a verified clean backup if needed, reapply roles, rotate

credentials, verify tenant isolation and release gradually with monitoring.

• **Learn:** publish an appropriately redacted post-incident review with owner,

corrective actions and measured detection/recovery times. Review controls and drills.

Private disclosure follows SECURITY.md. This runbook has not been exercised against a production incident or certified by an external assessor.

Implementation sources: docs/security/operations.md

30 Testing strategy and real results

Unit tests target calculations and validation; Spring HTTP and Testcontainers tests target identity, tenant/owner scope, idempotency and PostGIS constraints. Browser cases test routes, user controls, exports, mobile layouts, failures, accessibility and persisted state. Solidity tests exercise local authorization/replay/proof paths. Integration smoke calls actual services and checks persisted responses rather than replacing them with canned success.

The source manifest records measured revisions and workflow URLs. The baseline supports 29 Java tests with zero CI skips, 48 AI, 13 web unit, 18 browser, six ingestion, four contract tests and two mocked Terraform plans. Local Java skips seven Docker-dependent tests because Docker Desktop could not start; those skips are not local integration success. Linux CI supplies that evidence.

Compose validates forecasts, PulseAI evidence, saved scenarios, exports, idempotent data publication and restricted DB grants. Telemetry smoke verifies actual API/AI Prometheus scrape targets and spans stored in Tempo. Security jobs audit dependencies, scan secrets/source/configuration/images and produce CycloneDX inventories. Zero fixable HIGH/CRITICAL findings in the recorded images does not imply zero lower-severity or vendor-unfixed risk. A Hardhat development advisory remains disclosed.

These checks answer bounded engineering questions. They do not replace actual IdP/MFA enrollment, external penetration testing, a professional contract audit, field validation, manual accessibility review, representative saturation tests or recovery drills. No coverage percentage or production SLA is fabricated.

Implementation sources: .github/workflows/ci.yml; .github/workflows/security.yml; apps/web/e2e/theme.spec.ts

Check family	Recorded result	Evidence boundary
Java	29 passed, zero CI skips	Linux Testcontainers/PostGIS; local Docker tests skip
AI	48 passed	Forecast, prompt-injection, solver and telemetry tests
Web units	13 passed	Controls, sessions and streamed request limits
Browser	18 passed	Desktop/mobile; includes issue #22 theme regression
Ingestion	6 passed	Validation, deduplication, coordinates, time and calculations
Solidity	4 passed	Local chain; no professional contract audit
Terraform	2 mocked plans passed	No real AWS provisioning
Runtime images	Zero gated fixable HIGH/CRITICAL	Lower/unfixed findings outside this gate

Baseline CI: https://github.com/manikantbindass/CityPulse/actions/runs/40910819783. Theme CI: https://github.com/manikantbindass/CityPulse/actions/runs/40922082151. Security: https://github.com/manikantbindass/CityPulse/actions/runs/40922081880. These counts belong to recorded revisions, not future releases.

31 Demonstration scenarios and business applications

An ingestion walkthrough generates fixed-seed example-city rows, inspects normalized/quality/checkpoint/dead-letter output, publishes through the authorized API and replays. Acceptance is successful created-or-replayed behavior without duplicate writes, with provenance and times intact. A local checkpoint alone is not proof of remote publication.

An evidence walkthrough creates an owned PulseAI session and selects an assigned city, compatible commodity/unit/window. The rice fixture has demand without matching supply: the correct response explains why shortage cannot be established, cites allowed records and declines unsupported conclusions. A foreign session/tenant reference must not reveal content.

A numerical walkthrough trains the synthetic segments, reviews every candidate's validation/test results, activates an existing registry version and submits an actual forecast job. A planning walkthrough compares budget/capacity/floor/objective changes, preserves explicit infeasibility and distinguishes unsaved local results from Java-persisted owned scenarios. Export/reproduction refers to the evaluated input, not later form edits.

A proof walkthrough hashes the same exported bytes, submits an authorized local attestation, compares the receipt and changes one byte for a mismatch. The recorded gas of 150,788 applies to that local demonstration on chain 31337, not public-network fees. Blockchain protects a content reference, not measurement truth.

Potential value is reconstructable analysis for retail, logistics and public-service investigations. Real improvements in analyst time, service access, loss reduction or business profit have not been measured. A pilot must establish source completeness, outcome baselines, decision ownership and ethical/legal constraints before attributing benefits to the application.

Implementation sources: scripts/smoke.py; scripts/data_pipeline.py; docs/reports/blockchain-local-demo.json

32 Current limitations, future enhancements and conclusion

The primary limits are evidentiary and operational. Sources are synthetic unless separately configured and licensed. Missing coverage, overlapping providers, unequal periods and geographic definitions can invalidate apparent comparisons. Forecast errors are synthetic and one-step; residual intervals and horizon widening have no universal guarantee. Optimization assumes supplied costs/capacities and does not dispatch resources or derive routes.

Production identity, MFA, recovery and access reviews need actual provider validation. General business data needs individual-owner policy for mixed-owner tenants. Independent security review, Slither and restore exercises remain unperformed. Source deletion does not remove bytes already on a public chain or data retained in backups.

AI metadata uses SQLite/files, so a single writer and stop-before-start deployment are deliberate. EFS persistence is not distributed transaction safety. Durable workers, central job/model metadata, outbox/leases and object-store artifact integration remain future work. AWS modules are schema/mock-plan validated, not applied; private internal HTTP, broad NAT egress and writable ephemeral cloud roots remain documented deployment gaps.

The next meaningful milestone is a controlled pilot with licensed assessed data, verified ownership/identity boundaries, representative workload, recovery evidence and independent feedback. Scaling complexity should follow those measurements. CityPulse demonstrates disciplined integration and reproducibility, not certified production readiness, municipal adoption or guaranteed business success.

Implementation sources: docs/operations/scaling.md; docs/security/verification.md

33 Appendix A - API inventory

This static inventory extracts mapping annotations from Java controllers and route declarations from Python services. It is not runtime OpenAPI and does not imply public access. Read validation, service permissions and the actual authenticated contract for complete behavior.

Method	Static route	Declaration
GET	/api/v1/analytics/metrics	AnalyticsController.java
POST	/api/v1/verifications/{id}/receipt	ChainReceiptController.java
GET	/api/v1/cities	CityController.java
POST	/api/v1/decisions/networks	DecisionController.java
GET	/api/v1/decisions/scenarios	DecisionController.java

Method	Static route	Declaration
POST	/api/v1/decisions/scenarios	DecisionController.java
GET	/api/v1/decisions/scenarios/{id}	DecisionController.java
GET	/api/v1/decisions/scenarios/{id}/export	DecisionController.java
POST	/api/v1/decisions/scenarios/{id}/reproduce	DecisionController.java
POST	/api/v1/decisions/snapshots	DecisionController.java
POST	/api/v1/forecast-jobs	MlForecastController.java
GET	/api/v1/forecast-jobs/{id}	MlForecastController.java
GET	/api/v1/pulseai/cities	PulseAiController.java
GET	/api/v1/pulseai/sessions	PulseAiController.java
POST	/api/v1/pulseai/sessions	PulseAiController.java
DELETE	/api/v1/pulseai/sessions/{id}	PulseAiController.java
GET	/api/v1/pulseai/sessions/{id}	PulseAiController.java
POST	/api/v1/pulseai/sessions/{id}/messages	PulseAiController.java
GET	/api/v1/security/me	SecurityController.java
GET	/api/v1/admin/audit	WorkspaceController.java
GET	/api/v1/admin/configuration	WorkspaceController.java
PUT	/api/v1/admin/configuration	WorkspaceController.java
GET	/api/v1/analytics/imbalance	WorkspaceController.java
GET	/api/v1/areas	WorkspaceController.java
POST	/api/v1/areas	WorkspaceController.java
GET	/api/v1/businesses	WorkspaceController.java
POST	/api/v1/businesses	WorkspaceController.java
POST	/api/v1/cities	WorkspaceController.java
GET	/api/v1/cities/{id}/zones	WorkspaceController.java
GET	/api/v1/forecasts	WorkspaceController.java
POST	/api/v1/forecasts	WorkspaceController.java
POST	/api/v1/imports/observations	WorkspaceController.java
GET	/api/v1/inventories	WorkspaceController.java
POST	/api/v1/inventories	WorkspaceController.java
GET	/api/v1/notifications	WorkspaceController.java
POST	/api/v1/notifications	WorkspaceController.java
PATCH	/api/v1/notifications/{id}/read	WorkspaceController.java

Method	Static route	Declaration
GET	/api/v1/observations	WorkspaceController.java
POST	/api/v1/observations	WorkspaceController.java
GET	/api/v1/opportunities	WorkspaceController.java
POST	/api/v1/optimizations	WorkspaceController.java
GET	/api/v1/organizations	WorkspaceController.java
POST	/api/v1/organizations	WorkspaceController.java
GET	/api/v1/reports/export	WorkspaceController.java
GET	/api/v1/sources	WorkspaceController.java
POST	/api/v1/sources	WorkspaceController.java
GET	/api/v1/tenants/me	WorkspaceController.java
PATCH	/api/v1/tenants/me	WorkspaceController.java
GET	/api/v1/users	WorkspaceController.java
POST	/api/v1/users	WorkspaceController.java
PATCH	/api/v1/users/{id}/role	WorkspaceController.java
GET	/api/v1/verifications	WorkspaceController.java
POST	/api/v1/verifications	WorkspaceController.java
POST	/api/v1/zones	WorkspaceController.java
POST	/v1/decisions/evaluate	decisions.py
POST	/v1/forecasts	forecast_jobs.py
POST	/v1/forecasts/batch	forecast_jobs.py
GET	/v1/forecasts/{job_id}	forecast_jobs.py
GET	/v1/models	forecast_jobs.py
GET	/v1/models/{model_id}/metrics	forecast_jobs.py
POST	/api/v1/forecasts	main.py
GET	/health	main.py
POST	/v1/pulseai/select	pulseai.py
GET	/metrics	telemetry.py
GET	/ready	telemetry.py

Next adapters: /api/auth/[action], /api/cities, /api/evidence, /api/chain, /api/ml, /api/forecast, /api/pulseai/[[...path]], /api/decisions/[[...path]]. Health/management routes are deployment boundaries; OpenAPI does not grant access.

34 Appendix B - physical data dictionary

Physical columns are extracted from versioned CREATE TABLE and ALTER TABLE ADD COLUMN statements. This is a source inventory, not a live schema introspection. Constraints, triggers, grants and migration order remain authoritative in the cited SQL. Tables repeat headers and wrap long declarations.

city

Physical field	SQL declaration / constraints
id	uuid PRIMARY KEY
name	text NOT NULL CHECK (length(name) BETWEEN 1 AND 200)
country_code	char(2) NOT NULL CHECK (country_code ~ '^[A-Z]{2}$')
timezone	text NOT NULL
boundary	geometry(MultiPolygon,4326) NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()

zone

Physical field	SQL declaration / constraints
id	uuid PRIMARY KEY
city_id	uuid NOT NULL REFERENCES city(id)
name	text NOT NULL
boundary	geometry(MultiPolygon,4326) NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()

business

Physical field	SQL declaration / constraints
id	uuid PRIMARY KEY
city_id	uuid NOT NULL REFERENCES city(id)
zone_id	uuid NOT NULL
name	text NOT NULL
category	text NOT NULL
location	geometry(Point,4326) NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
tenant_id	uuid NOT NULL REFERENCES tenant(id)

source_metadata

Physical field	SQL declaration / constraints
id	uuid PRIMARY KEY
provider	text NOT NULL
dataset_name	text NOT NULL
source_uri	text NOT NULL
license_reference	text NOT NULL
classification	text NOT NULL CHECK (classification IN ('public','internal','restricted'))
schema_version	integer NOT NULL CHECK (schema_version > 0)
created_at	timestamptz NOT NULL DEFAULT now()
tenant_id	uuid NOT NULL REFERENCES tenant(id)
update_frequency	text NOT NULL DEFAULT 'unspecified'
quality_status	text NOT NULL DEFAULT 'unvalidated'
data_mode	text NOT NULL DEFAULT 'historical' CHECK(data_mode IN ('synthetic','historical','live'))
source_schema	jsonb NOT NULL DEFAULT '{"version":1}'::jsonb
timezone	text NOT NULL DEFAULT 'UTC'
currency	char(3) NOT NULL DEFAULT 'XXX' CHECK(currency ~ '^[A-Z]{3}$')

observation

Physical field	SQL declaration / constraints
id	uuid PRIMARY KEY
city_id	uuid NOT NULL REFERENCES city(id)
zone_id	uuid NOT NULL
business_id	uuid
source_id	uuid NOT NULL REFERENCES source_metadata(id)
external_id	text NOT NULL
kind	text NOT NULL CHECK (kind IN ('supply','demand'))
commodity	text NOT NULL
quantity	numeric(18,4) NOT NULL CHECK (quantity >= 0)
unit	text NOT NULL CHECK (unit IN ('count','kg','litre','kwh','trip'))
observed_at	timestamptz NOT NULL
ingested_at	timestamptz NOT NULL DEFAULT now()
period_end	timestamptz NOT NULL
provenance	jsonb NOT NULL CHECK (jsonb_typeof(provenance) = 'object')
payload_sha256	char(64) NOT NULL CHECK (payload_sha256 ~ '^[0-9a-f]{64}$')
quality_score	numeric(5,4) NOT NULL CHECK (quality_score BETWEEN 0 AND 1)
tenant_id	uuid NOT NULL REFERENCES tenant(id)

inventory

Physical field	SQL declaration / constraints
id	uuid PRIMARY KEY
business_id	uuid NOT NULL REFERENCES business(id)
source_id	uuid NOT NULL REFERENCES source_metadata(id)
external_id	text NOT NULL
commodity	text NOT NULL
quantity	numeric(18,4) NOT NULL CHECK (quantity >= 0)
unit	text NOT NULL CHECK (unit IN ('count','kg','litre','kwh','trip'))
recorded_at	timestamptz NOT NULL
ingested_at	timestamptz NOT NULL DEFAULT now()
provenance	jsonb NOT NULL CHECK (jsonb_typeof(provenance) = 'object')
tenant_id	uuid NOT NULL REFERENCES tenant(id)

audit_event

Physical field	SQL declaration / constraints
id	uuid PRIMARY KEY
actor_subject	text NOT NULL
action	text NOT NULL
resource_type	text NOT NULL
resource_id	uuid NOT NULL
occurred_at	timestamptz NOT NULL DEFAULT now()
trace_id	text
metadata	jsonb NOT NULL DEFAULT '{}'::jsonb
tenant_id	uuid NOT NULL REFERENCES tenant(id)

tenant

Physical field	SQL declaration / constraints
id	uuid PRIMARY KEY
name	varchar(120) NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()

organization

Physical field	SQL declaration / constraints
id	uuid PRIMARY KEY
tenant_id	uuid NOT NULL REFERENCES tenant(id)
name	varchar(120) NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()

user_account

Physical field	SQL declaration / constraints
id	uuid PRIMARY KEY
tenant_id	uuid NOT NULL REFERENCES tenant(id)
subject	varchar(200) NOT NULL
display_name	varchar(120) NOT NULL
role	varchar(30) NOT NULL CHECK(role IN ('ADMIN','ANALYST','PLANNER','BUSINESS_OWNER','PROVIDER','CITIZEN'))
enabled	boolean NOT NULL DEFAULT true
created_at	timestamptz NOT NULL DEFAULT now()

tenant_city

Physical field	SQL declaration / constraints
tenant_id	uuid NOT NULL REFERENCES tenant(id)
city_id	uuid NOT NULL REFERENCES city(id)

geographic_area

Physical field	SQL declaration / constraints
id	uuid PRIMARY KEY
tenant_id	uuid NOT NULL REFERENCES tenant(id)
city_id	uuid NOT NULL REFERENCES city(id)
parent_id	uuid
kind	varchar(20) NOT NULL CHECK(kind IN ('REGION','DISTRICT','NEIGHBORHOOD'))
name	varchar(120) NOT NULL
boundary	geometry(MultiPolygon,4326) NOT NULL

notification

Physical field	SQL declaration / constraints
id	uuid PRIMARY KEY
tenant_id	uuid NOT NULL REFERENCES tenant(id)
recipient_subject	varchar(200) NOT NULL
title	varchar(160) NOT NULL
message	varchar(1000) NOT NULL
read_at	timestamptz
created_at	timestamptz NOT NULL DEFAULT now()

operation_receipt

Physical field	SQL declaration / constraints
tenant_id	uuid NOT NULL REFERENCES tenant(id)
subject	varchar(200) NOT NULL
operation	varchar(80) NOT NULL
idempotency_key	varchar(100) NOT NULL
request_digest	char(64) NOT NULL
result	jsonb NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()

forecast_result

Physical field	SQL declaration / constraints
id	uuid PRIMARY KEY
tenant_id	uuid NOT NULL REFERENCES tenant(id)
request	jsonb NOT NULL
result	jsonb NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()

verification_request

Physical field	SQL declaration / constraints
id	uuid PRIMARY KEY
tenant_id	uuid NOT NULL REFERENCES tenant(id)
digest	char(64) NOT NULL
status	varchar(30) NOT NULL CHECK(status='DIGEST_RECORDED')
created_at	timestamptz NOT NULL DEFAULT now()
chain_id	bigint
contract_address	varchar(42)
contract_version	varchar(30)
transaction_hash	varchar(66)
block_hash	varchar(66)
block_number	bigint
attestation_id	varchar(66)
verified_at	timestamptz

admin_configuration

Physical field	SQL declaration / constraints
tenant_id	uuid PRIMARY KEY REFERENCES tenant(id)
configuration	jsonb NOT NULL DEFAULT '{}'::jsonb
updated_at	timestamptz NOT NULL DEFAULT now()

pulseai_session

Physical field	SQL declaration / constraints
id	uuid PRIMARY KEY
tenant_id	uuid NOT NULL REFERENCES tenant(id)
actor_subject	varchar(200) NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
expires_at	timestamptz NOT NULL DEFAULT now()+interval '7 days'

pulseai_message

Physical field	SQL declaration / constraints
id	uuid PRIMARY KEY
session_id	uuid NOT NULL REFERENCES pulseai_session(id) ON DELETE CASCADE
question	varchar(2000) NOT NULL
answer	jsonb NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()

decision_snapshot

Physical field	SQL declaration / constraints
id	uuid PRIMARY KEY
tenant_id	uuid NOT NULL REFERENCES tenant(id)
actor_subject	varchar(200) NOT NULL
payload	jsonb NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()

decision_network

Physical field	SQL declaration / constraints
id	uuid PRIMARY KEY
tenant_id	uuid NOT NULL REFERENCES tenant(id)
actor_subject	varchar(200) NOT NULL
source_id	uuid NOT NULL
payload	jsonb NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()

decision_scenario

Physical field	SQL declaration / constraints
id	uuid PRIMARY KEY
tenant_id	uuid NOT NULL REFERENCES tenant(id)
actor_subject	varchar(200) NOT NULL
name	varchar(120) NOT NULL
input	jsonb NOT NULL
result	jsonb NOT NULL
input_sha256	char(64) NOT NULL
result_sha256	char(64) NOT NULL
engine_version	varchar(80) NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()

35 Appendix C - architecture decision records

- 001 Modular application
- 002 PostGIS
- 003 Identity
- 004 AI and attestation
- 005 Events and infrastructure
- 006 Frontend
- 009: Progressive platform and AI writer boundary

Implementation sources: docs/architecture/decisions/README.md

Decision record	Context / recorded rationale
001-modular-application.md	Status: accepted Service fragmentation would introduce distributed transactions before demonstrated need. Keep bounded contexts in one Java deployment; isolate Python libraries in FastAPI.
002-postgis.md	Status: accepted Geographic relationships and relational constraints must agree. Use WGS84 geometry, GiST indexes, UTC event times, tenant predicates and SQL migrations.
003-identity.md	Status: accepted UI state cannot authorize protected actions. External IdP owns credentials. Validate issuer/audience and scope claims. Derive tenant from signed JWT, never a caller header. Use local random token only in local profile.

Decision record	Context / recorded rationale
004-ai-attestation.md	Status: accepted Forecast quality needs evidence; chain submission does not prove observation truth. Use named linear baseline and local submitter-scoped Solidity registry.
005-events-infrastructure.md	Status: accepted No production event flow currently justifies a broker or cluster. Use local PostGIS/Redis and no broker. Add database-backed jobs before an outbox/Redpanda.
006-frontend.md	Status: accepted One application serves marketing and workspace with explicit sample mode. Use App Router, Radix/shadcn-style controls, Query/Zustand, Recharts and lazy MapLibre/deck.gl.
007-pulseai-evidence-selection.md	Status: accepted for the local foundation. The analyst must remain useful without paid credentials and must not invent city claims. The Spring modular API owns authentication, fixed parameterized tools, canonical factual statements and citations. FastAPI abstracts an optional provider that selects allowed statement IDs using a strict schema. Java independently validates that selection. Source and document text never become executable tools. This limits conversational flexibility and prevents free-form model explanations. It gives deterministic fallback and a clear fact/citation boundary. Future richer responses require evaluated claim-level grounding and permission regression tests. Session storage stays in PostgreSQL, tenant+subject scoped, expiring after seven days; daily provider reservations use the existing persistent AI service volume. A distributed multi-replica provider budget requires a shared atomic ledger before horizontal AI scaling. No Kafka, vector database or new microservice is justified.
008-security-and-decisions.md	Status: accepted for the development foundation. Use the existing Python service with SciPy LP/MILP for bounded inventory transport, instead of introducing a scheduling microservice. Preserve observations, forecasts, inventory stocks and business assumptions separately. Store immutable tenant/owner snapshots and audited scenarios in PostgreSQL, with canonical fingerprints and explicit infeasibility. No Kafka or routing provider is justified without real data. Centralize all browser adapters on OIDC or an explicitly enabled local identity. Use a maintained OIDC client, PKCE/state/nonce and signature validation, then verify backend membership. Store encrypted short-lived access tokens server-side in Redis behind opaque Secure cookies; no refresh-token persistence. Scope and tenant checks remain independently enforced in Java. Configure provider MFA with verified ACR, not a local password/MFA implementation. Dynamic CSP nonces trade static rendering for a stronger script boundary. Inline style allowance remains for maps/charts.
009-progressive-platform.md	Status: accepted for development; cloud provisioning unvalidated. Keep Compose as the executable integration environment and ECS/Fargate as the first cloud path. Use private managed PostGIS/Redis, immutable image digests, separate migration secrets, a TLS ALB, encrypted persistent state and bounded web/API scaling. Service counts stay zero until operator bootstrap. Keep cloud apply outside CI, and validate Terraform using provider schemas and mocked plans. Use OpenTelemetry with allowlisted trace attributes, private Prometheus endpoints and explicit proposed SLOs. Local Prometheus/Grafana/Tempo provides inspectable signals without paid accounts. Do not claim alert delivery, a production restore or SLO attainment without evidence.

Historical ADRs record decisions at their original phase. Later implementation and ADR 009 supersede obsolete planned/implemented statements. Read the full editable ADRs in the repository for the historical context.

36 Appendix D - commands, configuration and glossary

Commands are developer/operator procedures, not proof that every environment has executed them. Secrets are not included. Reserved folders are boundaries rather than independent deployed applications.

Task	Command / prerequisite
Local stack	python scripts/dev_stack.py up --chain --observability; Docker/Compose and ignored .env
Safe stop	python scripts/dev_stack.py down --chain --observability; named volumes preserved
Web checks	npm ci; npm run lint; npm run typecheck; npm test --workspace apps/web
Browser suite	npm run build; npm run test:e2e --workspace apps/web; Chromium
API	mvn -f services/api/pom.xml verify; Java 21+; Docker for PostGIS tests
AI	python -m pytest services/ai-engine/tests; activated service environment
Ingestion	python scripts/data_pipeline.py demo; publish needs authorized API
Chain	npm run demo:ephemeral --workspace blockchain; public local fixture keys
Infrastructure	terraform init -backend=false; terraform validate; terraform test; no apply
Report	pip install -r docs/reports/source/requirements.txt; python scripts/build-report.py
Report checks	python scripts/check_report.py; optional PyMuPDF render inspection

Configuration inventory

Declared names in .env.example: POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD, REDIS_PASSWORD, LOCAL_API_TOKEN, WEB_PORT, API_PORT, AI_PORT, DATABASE_URL, REDIS_HOST, REDIS_PORT, AI_BASE_URL, OIDC_ISSUER_URI, OIDC_AUDIENCE, BLOCKCHAIN_RPC_URL, BLOCKCHAIN_CHAIN_ID, ATTESTATION_CONTRACT_ADDRESS, ATTESTATION_EVENT_TOPIC, ATTESTATION_CONTRACT_VERSION, TESTNET_RPC_URL, TESTNET_PRIVATE_KEY, CONTRACT_ADMIN_ADDRESS, PULSEAI_LOCAL_PREVIEW, PULSEAI_PROVIDER, PULSEAI_PROVIDER_URL, PULSEAI_MODEL, PULSEAI_API_KEY, PULSEAI_MAX_USD_PER_MILLION_TOKENS, DECISION_LOCAL_PREVIEW, DATABASE_USER, DATABASE_PASSWORD, FLYWAY_ENABLED, WEB_ORIGIN, OIDC_CLIENT_ID, OIDC_CLIENT_SECRET, OIDC_SCOPES, OIDC_REQUIRED_ACR, SESSION_REDIS_URL, SESSION_REDIS_ALLOW_LOCAL, SESSION_ENCRYPTION_KEY, GRAFANA_ADMIN_PASSWORD. Values are deliberately omitted.

Configuration boundary	Handling
OIDC issuer/audience/client	Operator IdP registration; client secret injected, never exposed to browser
Session Redis/encryption key	Private TLS store; random 32-byte base64 key; rotation invalidates sessions
Database runtime/migrator	Separate restricted runtime and schema-owner migration credentials
AI internal identity	Private service credential and server-derived tenant; no browser tenant trust
Optional provider	Disabled by default; explicit model/key and conservative cost bounds
Local flags/testnet	Never production defaults; public Hardhat test keys hold no real funds

Glossary

Term	Meaning
Attestation	Authorized digest reference, not source truth
BFF	Server adapter boundary between browser and API
C4	Context/container/component/deployment abstraction levels
CRS/SRID	Coordinate reference system / stored reference identifier
Flow / stock	Quantity over an event interval / inventory snapshot
Idempotency	Same scoped operation key and digest returns its recorded result
Leakage	Information unavailable at prediction origin entering a model or evaluation
MAE / RMSE / sMAPE	Metrics defined in modeling.py; synthetic results are not field accuracy
OIDC / PKCE / ACR	Identity protocol / code-exchange protection / assurance claim
RPO / RTO	Unverified recovery-point / recovery-time objectives
SLI / SLO	Measured indicator / proposed objective
Tenant / subject	Server-validated organization scope / authenticated actor
LP / MILP	Continuous / mixed-integer linear optimization
Unknown	Absent evidence, not an established zero

Directory boundaries

Root	Responsibility
apps/	Web application and admin extraction boundary
services/	Java API, Python AI and ingestion CLI; reporting boundary
packages/	Typed contracts/UI/config boundaries
blockchain/	Contracts, deployment/demo/verification scripts and tests
data/	Synthetic examples and schemas
infrastructure/	Docker, AWS Terraform, monitoring; optional Kubernetes planned
docs/	PRD, ADRs, API/security/operations, assets and report sources
tests/ and scripts/	Integration guidance, bounded load workloads and developer commands

37 References, attributions and reproducibility

Repository files are first-party implementation evidence. Citations identify inspected source paths at the recorded revision. External references support technology semantics, not deployment success. The font is Bitstream Vera from ReportLab; its package license permits embedding and no font files are separately committed. Project ownership is the verified GitHub account, not an inferred legal name, academic institution or student identifier. The repository has no owner-selected project license. The companion 33-slide project presentation is regenerated from the same inspected application snapshot and measured artifacts, with editable charts, diagrams and source-linked speaker notes.

[1] Repository source snapshot. https://github.com/manikantbindass/CityPulse/tree/2385ea7a3c6f9349c9c9ee67a56ace5f279fa992. Accessed 2026-10-29.

[2] Core CI verification. https://github.com/manikantbindass/CityPulse/actions/runs/40910819783. Accessed 2026-10-29.

[3] Theme issue and regression. https://github.com/manikantbindass/CityPulse/pull/23. Accessed 2026-10-29.

[4] Theme CI. https://github.com/manikantbindass/CityPulse/actions/runs/40922082151. Accessed 2026-10-29.

[5] Security and SBOM verification. https://github.com/manikantbindass/CityPulse/actions/runs/40922081880. Accessed 2026-10-29.

[6] Spring Boot tracing. https://docs.spring.io/spring-boot/4.0/reference/actuator/tracing.html. Accessed 2026-10-29.

[7] Next.js instrumentation. https://nextjs.org/docs/app/guides/instrumentation. Accessed 2026-10-29.

[8] Ridge regression API. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Ridge.html. Accessed 2026-10-29.

[9] SciPy mixed-integer linear programming. https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.milp.html. Accessed 2026-10-29.

[10] AWS RDS PostGIS management. https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Appendix.PostgreSQL.CommonDBATasks.PostGIS.html. Accessed 2026-10-29.

[11] AWS ECS deployment behavior. https://docs.aws.amazon.com/AmazonECS/latest/developerguide/deployment-type-ecs.html. Accessed 2026-10-29.

[12] k6 thresholds. https://grafana.com/docs/k6/latest/using-k6/thresholds/. Accessed 2026-10-29.

[13] ReportLab document composition. https://docs.reportlab.com/reportlab/userguide/ch5_platypus/. Accessed 2026-10-29.

[14] Fargate pricing inputs, not a quote. https://aws.amazon.com/fargate/pricing/. Accessed 2026-10-29.

[15] VPC cost inputs, not a quote. https://aws.amazon.com/vpc/pricing/. Accessed 2026-10-29.

Inspected repository evidence

Java source paths below abbreviate services/api/src/main/java/io/citypulse/ as api/. Full paths and hashes remain in evidence.json.

Inspected implementation source	Hash prefix
compose.yaml	d7c8bfa45fbf
.env.example	9d6f532ec216
apps/web/package.json	4e7a8f0a375d
apps/web/lib/server-auth.ts	feba3a2b3b71
apps/web/components/providers.tsx	d571f00cd416
apps/web/components/workspace-shell.tsx	7571240cdb2f
apps/web/e2e/theme.spec.ts	296c44f728f3
services/api/pom.xml	56843ec3c2d5
services/api/src/main/resources/application.yml	6e9cb088ff57
api/identity/SecurityConfiguration.java	bdd12dc20b78
api/identity/TokenPolicy.java	b2e94556444c
api/workspace/application/AnalyticsService.java	623eddc94930
api/decision/DecisionService.java	350135c139e5
api/pulseai/PulseAiService.java	1a3490b0931f
services/ai-engine/citypulse_ai/modeling.py	8ac7180908a8
services/ai-engine/citypulse_ai/forecast_jobs.py	bbdd848df19f
services/ai-engine/citypulse_ai/decisions.py	3df8ca4495c4
services/ai-engine/citypulse_ai/pulseai.py	1dfa8b7c6d31
services/ai-engine/citypulse_ai/telemetry.py	55e681ba7add
services/ingestion/citypulse_ingestion/pipeline.py	64e933a7532b
blockchain/contracts/AuthorizedSourceRegistry.sol	3ab9098aa7e0
blockchain/contracts/DatasetAttestationRegistry.sol	c8c101168992
blockchain/contracts/VerificationRecord.sol	e744e44b235f

Inspected implementation source	Hash prefix
docs/reports/blockchain-local-demo.json	fdf279b8f66d
docs/assets/readme-provenance.json	3a6bd565e615
infrastructure/terraform/main.tf.json	118ed0d4dace
infrastructure/terraform/modules/compute/main.tf.json	554fbd0ac9ab
infrastructure/terraform/modules/data/main.tf.json	edec93492374
infrastructure/monitoring/collector.yaml	8d09810144eb
.github/workflows/ci.yml	890ceb2a8f92
.github/workflows/security.yml	58d0dde8888c
.github/workflows/platform.yml	7be7cd574fac

Complete hashes are in source/evidence.json. Report generation is offline: no provider calls, cloud apply, signer operation or service mutation occurs.